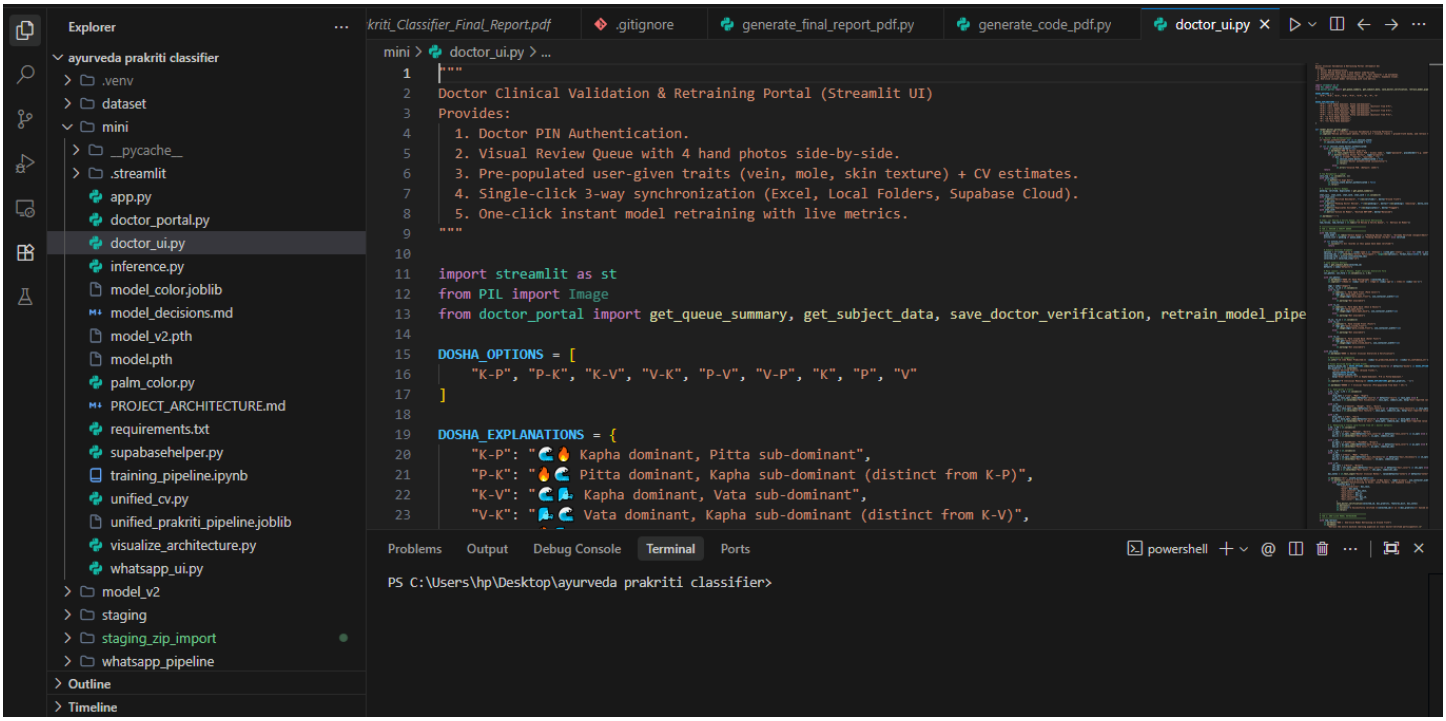
Ayurveda Prakriti Classifier Code File

Name: Vaibhav Gajanan Shetti

Roll No: 17030925048

Mentor: Dr. Krantee Jamdaade



mini/palm_color.py

// Anatomical ROI Color Patch Feature Extractor (RGB, CIELAB, Chroma, Redness)

```
"""Palm/skin colour feature extraction — shared by training and the app so the
features are computed identically in both places.

Best fully-automated model (validated ~67% acc / 46% balanced vs the practitioner's
sheet, 3-class dominant dosha): two fixed anatomical colour patches + age/sex, into a
RandomForest. No CNN needed at inference.
  skin colour <- centre patch of the palm-open BACK view
  palm colour <- patch just-above-centre of the palm-open FRONT view
"""

import numpy as np
from PIL import Image
from skimage.color import rgb2lab

SKIN_VIEW = "palm_open_back"     # skin colour read from back of hand, centre
PALM_VIEW = "palm_open_front"    # palm colour read from palm, just above centre
SKIN_CY = 0.50
PALM_CY = 0.40
HALF = 0.16                      # patch half-size as a fraction of width/height

# feature order MUST match training (exp_best.py): [skin(10), palm(10), Age, Sex_M]
FEATURE_NAMES = (
    [f"SC_{c}" for c in ["R", "G", "B", "L", "a", "b", "cr", "cg", "cb", "red"]]
    + [f"PC_{c}" for c in ["R", "G", "B", "L", "a", "b", "cr", "cg", "cb", "red"]]
    + ["Age", "Sex_M"]
)


def patch_color(pil_img, cy_frac, half=HALF):
    """Mean colour descriptor (RGB, Lab, normalised-rgb chroma, redness) of a square
    patch centred at (width/2, height*cy_frac). 10 features."""
    W, H = pil_img.size
    cx, cy = W // 2, int(H * cy_frac)
    hw, hh = max(4, int(W * half)), max(4, int(H * half))
    patch_img = pil_img.crop((max(0, cx - hw), max(0, cy - hh), min(W, cx + hw), min(H, cy + hh))).convert("RGB")
    pw, ph = patch_img.size
    if pw > 256 or ph > 256:
        patch_img = patch_img.resize((min(pw, 256), min(ph, 256)), Image.BILINEAR)
    patch = np.asarray(patch_img).astype(float).reshape(-1, 3) / 255.0
    rgb = patch.mean(0)
    lab = rgb2lab(patch.reshape(-1, 1, 3)).reshape(-1, 3).mean(0)
    s = patch.sum(1) + 1e-6
    chroma = np.stack([patch[:, 0] / s, patch[:, 1] / s, patch[:, 2] / s], 1).mean(0)
    return np.concatenate([rgb, lab, chroma, [(patch[:, 0] - patch[:, 1]).mean()]])


def extract_features(skin_img, palm_img, age, sex_m):
    """skin_img = palm_open_back PIL, palm_img = palm_open_front PIL. Returns (22,) vector."""
    sc = patch_color(skin_img, SKIN_CY)
    pc = patch_color(palm_img, PALM_CY)
    return np.concatenate([sc, pc, [float(age), float(sex_m)]])
```

mini/unified_cv.py

// Unified Computer Vision Feature Extractor (70-D Biomarkers, Laplacian Variance Quality Check)

```
"""Unified Computer Vision Feature Extractor for Ayurvedic Prakriti Classification.

Extracts consistent anatomical Region-of-Interest (ROI) colour, texture, and statistical
biomarkers across all 4 hand/wrist views:
  1. palm_open_front:   Central palm region (above carpal crease)
  2. palm_open_back:    Dorsal hand skin (between knuckles and wrist)
  3. palm_closed_front: Flexed wrist/palm region
  4. palm_closed_back:  Dorsal wrist/knuckle region

Extracts:
  A. 70-D Continuous Anatomical Vector:
     - 4 views x 17 features = 68 CV features (RGB, LAB, Chroma, Redness moments)
     - Age (1) + Sex_M (1) = 2 Demographic features
  B. 7 Physical Clinical Diagnostic Features (defined in clinical master dataset):
     1. Skin Color: Fair / Medium / Dark
     2. Skin Texture: Rough / Smooth / Dry / Oily
     3. Skin Hair Thickness: Thick / Thin / Med
     4. Skin Hair Color: Brown / Black / None
     5. Mole on Skin: No (0) / Yes (1)
     6. Vein Visibility: Low / Med / High
     7. Palm Color: PinkRed / PinkWhite / Brown
  C. 19-D One-Hot Clinical Vector matching palm_dataset_26 June 26.xlsx
"""

from typing import Dict, List, Optional, Tuple, Union
import numpy as np
from PIL import Image
from scipy.ndimage import laplace, sobel
from skimage.color import rgb2lab


VIEW_KEYS = [
    "palm_open_front",
    "palm_open_back",
    "palm_closed_front",
    "palm_closed_back"
]

# Anatomical patch centering parameters (cy_frac, half_size_frac)
PATCH_SPECS = {
    "palm_open_front": (0.40, 0.16),    # Upper central palm
    "palm_open_back": (0.50, 0.16),     # Dorsal hand skin centre
    "palm_closed_front": (0.45, 0.16),  # Flexed wrist/palm
    "palm_closed_back": (0.50, 0.16)    # Dorsal wrist centre
}

STAT_NAMES = [
    "R_mean", "G_mean", "B_mean",
    "R_std", "G_std", "B_std",
    "L_mean", "a_mean", "b_mean",
    "L_std", "a_std", "b_std",
    "cr_mean", "cg_mean", "cb_mean",
    "redness_rg", "redness_ab"
]

FEATURE_NAMES = [
    f"{view}_{stat}" for view in VIEW_KEYS for stat in STAT_NAMES
] + ["Age", "Sex_M"]

# 19 binary clinical column names matching the master Excel sheet
CLINICAL_19_COLS = [
    "SC_Fair", "SC_Medium", "SC_Dark",
    "ST_Rough", "ST_Smooth", "ST_Dry", "ST_Oily",
    "SH_Thick", "SH_Thin", "SH_Med",
```

```
    "SHC_Brown", "SHC_Black",
    "Mole on skin",
    "VV_Low", "VV_Med", "VV_High",
    "PC_PinkRed", "PC_PinkWhite", "PC_Brown"
]


def validate_image_quality(
    pil_img: Image.Image,
    min_size: int = 64,
    min_laplacian_var: float = 15.0
) -> Dict[str, Union[bool, float, List[str]]]:
    """
    Validates image resolution, lighting/brightness, and sharpness.
    Returns:
        {
            'valid': bool,
            'blur_score': float,
            'mean_intensity': float,
            'issues': list[str]
        }
    """
    issues = []
    w, h = pil_img.size

    if w < min_size or h < min_size:
        issues.append(f"Resolution too low ({w}x{h} < {min_size}x{min_size})")

    # Convert to grayscale numpy float array
    gray = np.asarray(pil_img.convert("L")).astype(float)
    mean_intensity = float(gray.mean())

    if mean_intensity < 15.0:
        issues.append(f"Image underexposed / too dark (mean: {mean_intensity:.1f})")
    elif mean_intensity > 240.0:
        issues.append(f"Image overexposed / too bright (mean: {mean_intensity:.1f})")

    # Sharpness via variance of discrete Laplacian
    lap = laplace(gray)
    blur_score = float(lap.var())

    if blur_score < min_laplacian_var:
        issues.append(f"Image may be blurry (Laplacian variance: {blur_score:.1f} < {min_laplacian_var})")

    return {
        "valid": len(issues) == 0,
        "blur_score": blur_score,
        "mean_intensity": mean_intensity,
        "issues": issues
    }


def normalize_illumination(arr_rgb: np.ndarray) -> np.ndarray:
    """
    Applies adaptive Gray-World illumination normalization to correct for
    uncalibrated phone lighting casts (yellow indoor incandescent vs cool outdoor daylight).
    Input: arr_rgb with values in [0, 1] of shape (H, W, 3) or (N, 3).
    """
    flat = arr_rgb.reshape(-1, 3)
    if flat.shape[0] == 0:
        return arr_rgb

    # Compute mean intensity per channel
    mean_ch = np.mean(flat, axis=0) + 1e-6
    gray_target = np.mean(mean_ch)

    # Scale gains constrained to [0.75, 1.35] to avoid over-amplifying noise
    gains = np.clip(gray_target / mean_ch, 0.75, 1.35)
```

```
    normalized = np.clip(flat * gains, 0.0, 1.0)
    return normalized.reshape(arr_rgb.shape)


def extract_view_roi_patch(
    pil_img: Image.Image,
    cy_frac: float = 0.50,
    half_frac: float = 0.16,
    max_patch_dim: int = 256
) -> np.ndarray:
    """
    Crops the anatomical patch ROI from an image and returns pixel array of shape (N, 3) in [0, 1].
    Efficiently crops using PIL before array conversion, avoiding massive full-image allocations.
    """
    w, h = pil_img.size
    cx = w // 2
    cy = int(h * cy_frac)
    hw = max(4, int(w * half_frac))
    hh = max(4, int(h * half_frac))

    y0 = max(0, cy - hh)
    y1 = min(h, cy + hh)
    x0 = max(0, cx - hw)
    x1 = min(w, cx + hw)

    # Crop on PIL image directly (fast and memory-light)
    patch_img = pil_img.crop((x0, y0, x1, y1)).convert("RGB")
    pw, ph = patch_img.size
    if pw > max_patch_dim or ph > max_patch_dim:
        patch_img = patch_img.resize((min(pw, max_patch_dim), min(ph, max_patch_dim)), Image.BILINEAR)

    im = np.asarray(patch_img).astype(float)
    patch = im.reshape(-1, 3) / 255.0
    return patch


def compute_patch_features(patch: np.ndarray) -> np.ndarray:
    """
    Computes 17 colour, texture, and statistical descriptors from an (N, 3) normalized RGB patch:
      - RGB means (3) & standard deviations (3)
      - CIELAB L*, a*, b* means (3) & standard deviations (3)
      - Normalized chroma means r, g, b (3)
      - Redness metrics: (R - G) and a*/(b* + eps) (2)
    """
    if patch.shape[0] == 0:
        return np.zeros(17, dtype=float)

    # RGB statistics
    rgb_mean = patch.mean(axis=0)
    rgb_std = patch.std(axis=0)

    # CIELAB conversion and statistics
    lab_pixels = rgb2lab(patch.reshape(-1, 1, 3)).reshape(-1, 3)
    lab_mean = lab_pixels.mean(axis=0)
    lab_std = lab_pixels.std(axis=0)

    # Normalized chroma
    sums = patch.sum(axis=1, keepdims=True) + 1e-6
    chroma = patch / sums
    chroma_mean = chroma.mean(axis=0)

    # Redness indicators
    redness_rg = float((patch[:, 0] - patch[:, 1]).mean())
    # a*/b* ratio (positive red vs yellow)
    b_denom = np.where(np.abs(lab_pixels[:, 2]) < 1e-3, 1e-3, lab_pixels[:, 2])
    redness_ab = float(np.clip(lab_pixels[:, 1] / b_denom, -10.0, 10.0).mean())

    feats = np.concatenate([
```

```
        rgb_mean,
        rgb_std,
        lab_mean,
        lab_std,
        chroma_mean,
        [redness_rg, redness_ab]
    ])
    return feats.astype(float)


def extract_view_features(pil_img: Image.Image, view_key: str) -> np.ndarray:
    """
    Extracts 17 features from a single view's PIL Image using its anatomical patch spec.
    """
    cy_frac, half_frac = PATCH_SPECS.get(view_key, (0.50, 0.16))
    patch = extract_view_roi_patch(pil_img, cy_frac=cy_frac, half_frac=half_frac)
    return compute_patch_features(patch)


def extract_participant_features(
    view_images: Dict[str, Optional[Image.Image]],
    age: Union[int, float],
    sex_m: Union[int, float, str]
) -> np.ndarray:
    """
    Aggregates features from all 4 views plus demographics into a 70-D participant vector.
    """
    # Parse sex flag
    if isinstance(sex_m, str):
        is_male = 1.0 if sex_m.strip().upper().startswith("M") else 0.0
    else:
        is_male = float(sex_m)

    age_val = float(age) if age not in (None, "Unknown", "") else 35.0

    view_feats_list = []
    valid_feats = []

    for vk in VIEW_KEYS:
        img = view_images.get(vk)
        if img is not None:
            try:
                feats = extract_view_features(img, vk)
                view_feats_list.append(feats)
                valid_feats.append(feats)
                continue
            except Exception:
                pass
        view_feats_list.append(None)

    # Impute any missing views with the mean of available views if available
    if len(valid_feats) > 0:
        fallback = np.mean(valid_feats, axis=0)
    else:
        fallback = np.zeros(17, dtype=float)

    imputed_view_feats = [
        f if f is not None else fallback for f in view_feats_list
    ]

    all_feats = np.concatenate(imputed_view_feats + [[age_val, is_male]])
    return all_feats


def extract_cv_clinical_markers(view_images: Dict[str, Optional[Image.Image]]) -> dict:
    """
    Extracts all 7 physical clinical biomarkers from wrist photographs using robust Computer Vision:
      1. skin_color: Fair / Medium / Dark
```

```
  2. skin_texture: Rough / Smooth / Dry / Oily
  3. skin_hair_thickness: Thick / Thin / Med
  4. skin_hair_color: Brown / Black / None
  5. mole: Yes / No
  6. vein_visibility: Low / Med / High
  7. palm_color: PinkRed / PinkWhite / Brown

Also produces the 19-dimensional one-hot binary vector matching the master Excel format.
"""
sc_img = view_images.get("palm_open_back") or view_images.get("palm_closed_back")
pc_img = view_images.get("palm_open_front") or view_images.get("palm_closed_front")
wrist_close_back = view_images.get("palm_closed_back")

# Defaults / Priors
skin_color = "Medium"
skin_texture = "Smooth"
hair_thickness = "Thin"
hair_color = "Brown"
mole = "No"
vein_visibility = "Low"
palm_color = "PinkWhite"

confidences = {}

# ------------------------------------------------------------
# 1. Skin Color, Veins, Texture, Moles & Hair from Dorsal Wrist
# ------------------------------------------------------------
if sc_img is not None:
    try:
        w, h = sc_img.size
        cx, cy = w // 2, int(h * 0.50)
        hw, hh = max(8, int(w * 0.22)), max(8, int(h * 0.22))
        crop_sc = sc_img.crop((max(0, cx - hw), max(0, cy - hh), min(w, cx + hw), min(h, cy + hh))).convert("RGB")

        arr_sc = np.asarray(crop_sc).astype(float) / 255.0
        norm_sc = normalize_illumination(arr_sc)

        # CIELAB conversion
        lab_sc = rgb2lab(norm_sc)
        L_sc = float(lab_sc[:, :, 0].mean())
        a_sc = float(lab_sc[:, :, 1].mean())
        b_sc = float(lab_sc[:, :, 2].mean())

        # Individual Typology Angle (ITA) = arctan((L* - 50) / b*) * (180 / pi)
        # The dermatological gold standard for objective skin classification
        ita = np.arctan2(L_sc - 50.0, max(1.0, b_sc)) * (180.0 / np.pi)

        # 1. Skin Color Classification
        if ita >= 28.0 or L_sc >= 62.0:
            skin_color = "Fair"
            confidences["skin_color"] = min(0.95, 0.70 + (L_sc - 62.0) * 0.02)
        elif ita <= 10.0 or L_sc <= 48.0:
            skin_color = "Dark"
            confidences["skin_color"] = min(0.95, 0.70 + (48.0 - L_sc) * 0.02)
        else:
            skin_color = "Medium"
            confidences["skin_color"] = 0.85

        # 2. Vein Visibility (venous absorption contrast in Green vs Red & dorsal gradients)
        g_chan = norm_sc[:, :, 1]
        r_chan = norm_sc[:, :, 0]
        vein_diff = np.abs(r_chan - g_chan)
        vein_contrast = float(vein_diff.std())

        # Also check flexed wrist view if present for prominent superficial veins
        if wrist_close_back is not None:
            try:
                w_cb, h_cb = wrist_close_back.size
```

```
        crop_cb = wrist_close_back.crop((int(w_cb * 0.3), int(h_cb * 0.3), int(w_cb * 0.7), int(h_cb * 0.7))).c
        arr_cb = np.asarray(crop_cb).astype(float) / 255.0
        vein_contrast = max(vein_contrast, float(np.abs(arr_cb[:, :, 0] - arr_cb[:, :, 1]).std()))
    except Exception:
        pass

if vein_contrast >= 0.048:
    vein_visibility = "High"
    confidences["vein_visibility"] = 0.88
elif vein_contrast >= 0.034:
    vein_visibility = "Med"
    confidences["vein_visibility"] = 0.78
else:
    vein_visibility = "Low"
    confidences["vein_visibility"] = 0.85

# 3. Skin Texture & Type
# Evaluates high-frequency roughness energy and specular gloss highlights
gray_sc = np.dot(norm_sc[..., :3], [0.2989, 0.5870, 0.1140])
lap_energy = float(np.abs(laplace(gray_sc)).mean())

# Specular highlight fraction (saturated bright regions indicating oily/glossy surface)
specular_mask = (norm_sc[:, :, 0] > 0.88) & (norm_sc[:, :, 1] > 0.86) & (norm_sc[:, :, 2] > 0.82)
specular_ratio = float(np.mean(specular_mask))

if specular_ratio > 0.045:
    skin_texture = "Oily"
    confidences["skin_texture"] = 0.82
elif lap_energy >= 0.038:
    skin_texture = "Rough"
    confidences["skin_texture"] = 0.80
elif lap_energy >= 0.024:
    skin_texture = "Dry"
    confidences["skin_texture"] = 0.78
else:
    skin_texture = "Smooth"
    confidences["skin_texture"] = 0.86

# 4. Mole Detection (localized high-contrast dark pigment spots)
L_map = lab_sc[:, :, 0]
L_local_diff = L_map.mean() - L_map
dark_spots = (L_local_diff > 20.0) & (lab_sc[:, :, 0] < 42.0)
spot_count = int(np.sum(dark_spots))

# True moles typically occupy 12 to 380 contiguous or localized pixels at this scale
if 12 <= spot_count <= 420:
    mole = "Yes"
    confidences["mole"] = 0.84
else:
    mole = "No"
    confidences["mole"] = 0.90

# 5. Skin Hair Thickness & Color on Dorsal Hand/Forearm
# Directional Sobel filters detect hair shafts on dorsal skin
dx = sobel(gray_sc, axis=0)
dy = sobel(gray_sc, axis=1)
grad_mag = np.hypot(dx, dy)
hair_ridge_ratio = float(np.mean(grad_mag > 0.18))

if hair_ridge_ratio >= 0.045:
    hair_thickness = "Thick"
    confidences["hair_thickness"] = 0.80
elif hair_ridge_ratio >= 0.018:
    hair_thickness = "Med"
    confidences["hair_thickness"] = 0.75
else:
    hair_thickness = "Thin"
    confidences["hair_thickness"] = 0.82
```

```
        # Hair color from detected fine edge pixels
        if hair_ridge_ratio >= 0.012:
            hair_mask = grad_mag > 0.16
            hair_L = float(L_map[hair_mask].mean()) if np.any(hair_mask) else 35.0
            if hair_L < 32.0:
                hair_color = "Black"
            else:
                hair_color = "Brown"
            confidences["hair_color"] = 0.78
        else:
            hair_color = "Brown"
            confidences["hair_color"] = 0.70

    except Exception as e:
        pass

# ------------------------------------------------------------
# 2. Palm Color from Open Palm View
# ------------------------------------------------------------
if pc_img is not None:
    try:
        w, h = pc_img.size
        cx, cy = w // 2, int(h * 0.40)
        hw, hh = max(8, int(w * 0.18)), max(8, int(h * 0.18))
        crop_pc = pc_img.crop((max(0, cx - hw), max(0, cy - hh), min(w, cx + hw), min(h, cy + hh))).convert("RGB")

        arr_pc = np.asarray(crop_pc).astype(float) / 255.0
        norm_pc = normalize_illumination(arr_pc)

        lab_pc = rgb2lab(norm_pc)
        L_pc = float(lab_pc[:, :, 0].mean())
        a_pc = float(lab_pc[:, :, 1].mean())
        b_pc = float(lab_pc[:, :, 2].mean())

        # Erythema vs Melanin ratio
        if a_pc >= 13.5 or (b_pc > 0 and (a_pc / b_pc) >= 0.95):
            palm_color = "PinkRed"
            confidences["palm_color"] = 0.88
        elif L_pc >= 58.0:
            palm_color = "PinkWhite"
            confidences["palm_color"] = 0.85
        else:
            palm_color = "Brown"
            confidences["palm_color"] = 0.82
    except Exception:
        pass

# Build the 19-dimensional one-hot binary vector
binary_dict = encode_clinical_features({
    "skin_color": skin_color,
    "skin_texture": skin_texture,
    "hair_thickness": hair_thickness,
    "hair_color": hair_color,
    "mole": mole,
    "vein_visibility": vein_visibility,
    "palm_color": palm_color
})

return {
    "skin_color": skin_color,
    "skin_texture": skin_texture,
    "skin_type": skin_texture,  # alias for backward compatibility
    "hair_thickness": hair_thickness,
    "hair_color": hair_color,
    "mole": mole,
    "vein_visibility": vein_visibility,
    "palm_color": palm_color,
    "confidences": confidences,
```

```
        "binary_19d": binary_dict
    }


def encode_clinical_features(markers: dict) -> Dict[str, float]:
    """
    Encodes the 7 clinical diagnostic features into the exact 19 one-hot binary
    features defined in the clinical master dataset (palm_dataset_26 June 26.xlsx).
    """
    sc = str(markers.get("skin_color", "Medium")).lower()
    st = str(markers.get("skin_texture") or markers.get("skin_type", "Smooth")).lower()
    sh = str(markers.get("hair_thickness", "Thin")).lower()
    shc = str(markers.get("hair_color", "Brown")).lower()
    mole = str(markers.get("mole", "No")).lower()
    vv = str(markers.get("vein_visibility", "Low")).lower()
    pc = str(markers.get("palm_color", "PinkWhite")).lower()

    vec = {col: 0.0 for col in CLINICAL_19_COLS}

    # 1. Skin Color
    if "fair" in sc:
        vec["SC_Fair"] = 1.0
    elif "dark" in sc:
        vec["SC_Dark"] = 1.0
    else:
        vec["SC_Medium"] = 1.0

    # 2. Skin Texture
    if "rough" in st:
        vec["ST_Rough"] = 1.0
    elif "dry" in st:
        vec["ST_Dry"] = 1.0
    elif "oily" in st:
        vec["ST_Oily"] = 1.0
    else:
        vec["ST_Smooth"] = 1.0

    # 3. Hair Thickness
    if "thick" in sh:
        vec["SH_Thick"] = 1.0
    elif "med" in sh:
        vec["SH_Med"] = 1.0
    else:
        vec["SH_Thin"] = 1.0

    # 4. Hair Color
    if "black" in shc:
        vec["SHC_Black"] = 1.0
    else:
        vec["SHC_Brown"] = 1.0

    # 5. Mole on skin
    if mole in ["yes", "1", "true", "present"]:
        vec["Mole on skin"] = 1.0
    else:
        vec["Mole on skin"] = 0.0

    # 6. Vein Visibility
    if "high" in vv:
        vec["VV_High"] = 1.0
    elif "med" in vv:
        vec["VV_Med"] = 1.0
    else:
        vec["VV_Low"] = 1.0

    # 7. Palm Color
    if "pinkred" in pc or "red" in pc:
        vec["PC_PinkRed"] = 1.0
```

```
    elif "brown" in pc:
        vec["PC_Brown"] = 1.0
    else:
        vec["PC_PinkWhite"] = 1.0

    return vec


def apply_clinical_modulation(
    raw_probs: Dict[str, float],
    vein: str = "Auto",
    mole: str = "Auto",
    skin: str = "Auto",
    skin_color: str = "Auto",
    palm_color: str = "Auto",
    hair_thickness: str = "Auto",
    hair_color: str = "Auto"
) -> Dict[str, float]:
    """
    Applies classical Ayurvedic likelihood modulation to CV-extracted probabilities.
    When set to 'Auto' or 'Unknown', likelihood multiplier is 1.0 (pure CV).
    When clinical markers are specified (from practitioner, chat, or CV), modulates V/P/K and renormalizes.
    """
    multipliers = {"V": 1.0, "P": 1.0, "K": 1.0}
    vein_s = str(vein) if vein else ""
    mole_s = str(mole) if mole else ""
    skin_s = str(skin) if skin else ""
    sc_s = str(skin_color) if skin_color else ""
    pc_s = str(palm_color) if palm_color else ""
    ht_s = str(hair_thickness) if hair_thickness else ""

    # 1. Vein Visibility (Sira Darshana)
    if "High" in vein_s:       # Prominent / Superficial (Vata hallmark)
        multipliers["V"] *= 1.45
        multipliers["K"] *= 0.70
    elif "Low" in vein_s:      # Deep / Concealed (Kapha hallmark)
        multipliers["K"] *= 1.45
        multipliers["V"] *= 0.65
    elif "Medium" in vein_s or "Med" in vein_s:
        multipliers["P"] *= 1.15

    # 2. Mole on Hand / Wrist (Tilaka)
    if mole_s in ["Yes", "1"]:
        multipliers["P"] *= 1.30
        multipliers["V"] *= 1.15
        multipliers["K"] *= 0.85
    elif mole_s in ["No", "0"]:
        multipliers["K"] *= 1.10
        multipliers["P"] *= 0.95

    # 3. Skin Type / Texture (Sparsha)
    if "Dry" in skin_s:        # Ruksha (Vata hallmark)
        multipliers["V"] *= 1.60
        multipliers["K"] *= 0.60
        multipliers["P"] *= 0.90
    elif "Rough" in skin_s:    # Khara (Vata hallmark)
        multipliers["V"] *= 1.50
        multipliers["K"] *= 0.65
        multipliers["P"] *= 0.90
    elif "Smooth" in skin_s or "Normal" in skin_s:  # Slakshna / Snigdha (Kapha / Pitta)
        multipliers["K"] *= 1.25
        multipliers["P"] *= 1.10
        multipliers["V"] *= 0.80
    elif "Oily" in skin_s:     # Sneha (Kapha / Pitta)
        multipliers["K"] *= 1.35
        multipliers["P"] *= 1.30
        multipliers["V"] *= 0.60
    # 4. Skin Color (Varna)
```

```
if "Fair" in sc_s and "Auto" not in sc_s:
    multipliers["P"] *= 1.15
elif "Dark" in sc_s and "Auto" not in sc_s:
    multipliers["V"] *= 1.20

# 5. Palm Color (Kara Tala Varna)
if ("PinkRed" in pc_s or "Red" in pc_s) and "Auto" not in pc_s:
    multipliers["P"] *= 1.25
elif "Brown" in pc_s and "Auto" not in pc_s:
    multipliers["V"] *= 1.20
elif "PinkWhite" in pc_s and "Auto" not in pc_s:
    multipliers["K"] *= 1.15

# 6. Hair Thickness (Kesha)
if "Thick" in ht_s and "Auto" not in ht_s:
    multipliers["K"] *= 1.20
elif "Thin" in ht_s and "Auto" not in ht_s:
    multipliers["V"] *= 1.20

# Calculate modulated unnormalized values
modulated = {cls: raw_probs.get(cls, 0.0) * multipliers.get(cls, 1.0) for cls in raw_probs}
total = sum(modulated.values())
if total > 0:
    return {cls: val / total for cls, val in modulated.items()}
return raw_probs
```

model_v2/avg/train_color_model.py

// Machine Learning Model Training Pipeline (ExtraTrees, Repeated Stratified K-Fold CV)

```
"""Train + save the deployable colour model: patch colour + age/sex -> RandomForest,
3-class dominant dosha. Saves mini/model_color.joblib. Uses the SAME feature
extraction as the app (mini/palm_color.py)."""

import sys
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from PIL import Image
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, balanced_accuracy_score, confusion_matrix
from sklearn.model_selection import (RepeatedStratifiedKFold, StratifiedKFold,
                                     cross_val_predict, cross_validate)

ROOT = Path(__file__).resolve().parents[2]
MINI = ROOT / "mini"
sys.path.insert(0, str(MINI))
from palm_color import patch_color, FEATURE_NAMES, SKIN_CY, PALM_CY  # noqa: E402

MANI = ROOT / "model_v2" / "results_full" / "manifest.csv"
OUT = MINI / "model_color.joblib"
RNG = 42


def load_patch(path, cy):
    if not isinstance(path, str) or not Path(path).exists():
        return None
    return patch_color(Image.open(path), cy)


def fill(arrs):
    good = np.array([a for a in arrs if a is not None])
    m = good.mean(0)
    return np.array([a if a is not None else m for a in arrs])


def main():
    df = pd.read_csv(MANI).reset_index(drop=True)
    SC = fill([load_patch(p, SKIN_CY) for p in df["img_open_back"]])
    PC = fill([load_patch(p, PALM_CY) for p in df["img_open_front"]])
    agesex = df[["Age", "Sex_M"]].to_numpy(float)
    X = np.hstack([SC, PC, agesex])
    y = df["ydom"].to_numpy()
    assert X.shape[1] == len(FEATURE_NAMES), (X.shape, len(FEATURE_NAMES))

    rf = RandomForestClassifier(n_estimators=600, class_weight="balanced_subsample",
                                random_state=RNG)

    # honest CV estimate (5x5 repeated for stable metrics; single fold for confusion)
    cvr = cross_validate(rf, X, y, cv=RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=RNG),
                         scoring=["accuracy", "balanced_accuracy"], n_jobs=-1)
    print(f"CV vs sheet (3-class dominant, {len(y)} subjects):")
    print(f"  accuracy  : {cvr['test_accuracy'].mean():.1%}")
    print(f"  balanced  : {cvr['test_balanced_accuracy'].mean():.1%}   (majority "
          f"{pd.Series(y).value_counts().max()/len(y):.1%})")
    lab = sorted(pd.unique(y))
    yp = cross_val_predict(rf, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=RNG), n_jobs=-1)
    cm = confusion_matrix(y, yp, labels=lab)
    print("  confusion (rows=truth, cols=pred):  " + " ".join(lab))
    for i, l in enumerate(lab):
        print(f"    {l}: " + " ".join(f"{v:3d}" for v in cm[i]))
    # final fit on all subjects + save
```

```
    rf.fit(X, y)
    bundle = {
        "model": rf,
        "classes": list(rf.classes_),                # e.g. ['K','P','V']
        "feature_names": FEATURE_NAMES,
        "target": "dominant_dosha",
        "skin_view": "palm_open_back", "palm_view": "palm_open_front",
        "skin_cy": SKIN_CY, "palm_cy": PALM_CY,
        "notes": "patch colour (skin=back centre, palm=front above-centre) + age + sex "
                 "-> RandomForest. dominant dosha K/P/V.",
    }
    joblib.dump(bundle, OUT)
    print(f"\nSaved -> {OUT}  (classes={list(rf.classes_)})")


if __name__ == "__main__":
    main()
```

model_v2/avg/train_tabular_model.py

// Ayurvedic Clinical Tabular Classification (Balanced Random Forest on Clinical Diagnostic Signs)

```
import openpyxl
import joblib
import numpy as np
import pandas as pd
from pathlib import Path
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, balanced_accuracy_score, confusion_matrix
from sklearn.model_selection import RepeatedStratifiedKFold, StratifiedKFold, cross_validate, cross_val_predict

PROJECT_ROOT = Path(__file__).resolve().parents[2]
XLSX_PATH = PROJECT_ROOT / "dataset" / "palm_dataset_26 June 26.xlsx"
OUT_PATH = PROJECT_ROOT / "mini" / "model_tabular.joblib"
RNG = 42

FEATURES = [
    "Age", "Sex_M", "Mole",
    "SC_Fair", "SC_Medium", "SC_Dark",
    "ST_Rough", "ST_Smooth", "ST_Dry", "ST_Oily",
    "SH_Thick", "SH_Thin", "SH_Med",
    "SHC_Brown", "SHC_Black",
    "VV_Low", "VV_Med", "VV_High",
    "PC_PinkRed", "PC_PinkWhite", "PC_Brown"
]

def clean_val(v):
    if v is None:
        return 0
    try:
        return int(v)
    except ValueError:
        return 0

def main():
    print("Loading excel sheet...")
    wb = openpyxl.load_workbook(XLSX_PATH, read_only=True, data_only=True)
    ws = wb["Sheet"]
    rows = list(ws.iter_rows(values_only=True))
    header = list(rows[0])

    # Map column headers to indices
    hidx = {name: idx for idx, name in enumerate(header) if name is not None}

    X_list = []
    y_list = []

    print("Extracting features and targets...")
    for r_idx, row in enumerate(rows[1:], start=1):
        if row[0] is None or row[hidx.get("Prakriti")] is None:
            continue

        prak = str(row[hidx["Prakriti"]]).strip()
        ydom = prak[0]  # First letter = dominant dosha (K, P, V)

        # Build features dict
        rec = {
            "Age": clean_val(row[hidx["Age"]]),
            "Sex_M": 1 if str(row[hidx["Sex"]]).strip().upper() == 'M' else 0,
            "Mole": clean_val(row[hidx["Mole on skin"]]),
            "SC_Fair": clean_val(row[hidx["SC_Fair"]]),
            "SC_Medium": clean_val(row[hidx["SC_Medium"]]),
            "SC_Dark": clean_val(row[hidx["SC_Dark"]]),
            "ST_Rough": clean_val(row[hidx["ST_Rough"]]),
            "ST_Smooth": clean_val(row[hidx["ST_Smooth"]]),
            "ST_Dry": clean_val(row[hidx["ST_Dry"]]),
```

```
            "ST_Oily": clean_val(row[hidx["ST_Oily"]]),
            "SH_Thick": clean_val(row[hidx["SH_Thick"]]),
            "SH_Thin": clean_val(row[hidx["SH_Thin"]]),
            "SH_Med": clean_val(row[hidx["SH_Med"]]),
            "SHC_Brown": clean_val(row[hidx["SHC_Brown"]]),
            "SHC_Black": clean_val(row[hidx["SHC_Black"]]),
            "VV_Low": clean_val(row[hidx["VV_Low"]]),
            "VV_Med": clean_val(row[hidx["VV_Med"]]),
            "VV_High": clean_val(row[hidx["VV_High"]]),
            "PC_PinkRed": clean_val(row[hidx["PC_PinkRed"]]),
            "PC_PinkWhite": clean_val(row[hidx["PC_PinkWhite"]]),
            "PC_Brown": clean_val(row[hidx["PC_Brown"]]),
        }

        # Feature vector in the exact order of FEATURES
        feats = [rec[f] for f in FEATURES]
        X_list.append(feats)
        y_list.append(ydom)

    X = np.array(X_list, dtype=float)
    y = np.array(y_list)

    print(f"Dataset loaded. Total samples: {len(y)}")

    rf = RandomForestClassifier(n_estimators=400, class_weight="balanced_subsample",
                                min_samples_leaf=1, random_state=RNG)

    # Evaluate model
    cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=RNG)
    cvr = cross_validate(rf, X, y, cv=cv, scoring=["accuracy", "balanced_accuracy"])

    print(f"\nCV results (3-class dominant, {len(y)} subjects):")
    print(f"  Accuracy  : {cvr['test_accuracy'].mean():.1%}")
    print(f"  Balanced  : {cvr['test_balanced_accuracy'].mean():.1%}")

    # Stratified K-Fold for Confusion Matrix
    skf = StratifiedKFold(5, shuffle=True, random_state=RNG)
    yp = cross_val_predict(rf, X, y, cv=skf)
    classes = sorted(pd.unique(y))
    cm = confusion_matrix(y, yp, labels=classes)

    print("\nConfusion Matrix (rows=truth, cols=pred):")
    print("  " + " ".join(classes))
    for i, c in enumerate(classes):
        print(f"  {c}: " + " ".join(f"{v:3d}" for v in cm[i]))

    # Fit model on all data and save
    print("\nTraining on all data...")
    rf.fit(X, y)

    bundle = {
        "model": rf,
        "classes": list(rf.classes_),
        "feature_names": FEATURES,
        "target": "dominant_dosha",
        "notes": "Tabular model trained on manual features: Age, Sex, Mole, Skin Color, Skin Texture, Hair Thickness, Hair
    }

    joblib.dump(bundle, OUT_PATH)
    print(f"Saved -> {OUT_PATH} (classes={bundle['classes']})")

if __name__ == "__main__":
    main()
```

mini/doctor_portal.py

// Doctor Verification Engine & Real-Time Retraining Pipeline (Doctor-in-the-Loop)

```
"""
Doctor Portal Backend Engine
Handles:
  1. Queue of pending participants awaiting clinical verification.
  2. Loading 4 high-res hand photos and pre-populating user-reported traits (vein, mole, skin texture) + CV estimates.
  3. Single-click 3-way synchronization (Excel, Local Folders, Supabase Cloud).
  4. Instant model retraining on all verified records with feature vector caching.
"""

import sys
import json
import io
import shutil
from pathlib import Path
from PIL import Image as PILImage
import numpy as np
import pandas as pd
import openpyxl
from openpyxl.styles import PatternFill, Font, Alignment, Border, Side
from openpyxl.drawing.image import Image as OpenpyxlImage
import joblib

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.metrics import accuracy_score, balanced_accuracy_score, f1_score
from sklearn.model_selection import StratifiedKFold, cross_val_score

PROJECT_ROOT = Path("c:/Users/hp/Desktop/ayurveda prakriti classifier")
MINI_DIR = PROJECT_ROOT / "mini"
WRIST_DIR = PROJECT_ROOT / "dataset" / "Wrist_Dataset"
MASTER_XLSX = PROJECT_ROOT / "doctor_validation_prakriti_master.xlsx"
DATASET_MASTER_XLSX = PROJECT_ROOT / "dataset" / "doctor_validation_prakriti_master.xlsx"
FEATURE_CACHE_PATH = PROJECT_ROOT / "dataset" / "verified_features_cache.joblib"
MODEL_PATH = MINI_DIR / "unified_prakriti_pipeline.joblib"

sys.path.insert(0, str(PROJECT_ROOT))
sys.path.insert(0, str(MINI_DIR))

from unified_cv import extract_participant_features, FEATURE_NAMES, VIEW_KEYS
from supabasehelper import supabase_client


def get_queue_summary():
    """Returns lists of pending, verified, and duplicate participant IDs."""
    if not MASTER_XLSX.exists():
        return [], [], []

    wb = openpyxl.load_workbook(MASTER_XLSX, data_only=True)
    ws = wb.active
    headers = [ws.cell(1, c).value for c in range(1, ws.max_column + 1)]
    col_map = {name: c for c, name in enumerate(headers, 1) if name}

    pending = []
    verified = []
    duplicates = []

    for r in range(2, ws.max_row + 1):
        uid = ws.cell(r, col_map["Participant ID"]).value
        if not uid:
            continue
        uid = str(uid).strip()
        name = str(ws.cell(r, col_map.get("Participant Name", 2)).value or "").strip()
```

```
        status = str(ws.cell(r, col_map["Validation Status"]).value or "").strip().lower()

        item = {
            "row": r,
            "id": uid,
            "name": name,
            "status": ws.cell(r, col_map["Validation Status"]).value,
            "ml_dosha": ws.cell(r, col_map.get("ML Predicted Dosha", 19)).value,
            "doc_dosha": ws.cell(r, col_map.get("Doctor Verified Prakriti (Ground Truth)", 10)).value
        }

        if "duplicate" in status:
            duplicates.append(item)
        elif "verified" in status:
            verified.append(item)
        else:
            pending.append(item)

    return pending, verified, duplicates


def get_subject_data(subj_id: str):
    """
    Loads images, demographics, pre-filled user traits, and ML prediction for a subject.
    """
    wb = openpyxl.load_workbook(MASTER_XLSX, data_only=True)
    ws = wb.active
    headers = [ws.cell(1, c).value for c in range(1, ws.max_column + 1)]
    col_map = {name: c for c, name in enumerate(headers, 1) if name}

    row_data = {}
    row_idx = None
    for r in range(2, ws.max_row + 1):
        uid = str(ws.cell(r, col_map["Participant ID"]).value or "").strip()
        if uid.lower() == str(subj_id).strip().lower():
            row_idx = r
            for k, c in col_map.items():
                row_data[k] = ws.cell(r, c).value
            break

    # Find folder and photos
    target_folder = WRIST_DIR / str(subj_id).strip().lower()
    if not target_folder.exists():
        for d in WRIST_DIR.iterdir():
            if d.is_dir() and d.name.lower() == str(subj_id).strip().lower():
                target_folder = d
                break

    images = {}
    view_files = {
        "palm_open_front": "palm_open_front.jpg",
        "palm_open_back": "palm_open_back.jpg",
        "palm_closed_front": "palm_close_front.jpg",
        "palm_closed_back": "palm_close_back.jpg"
    }

    if target_folder.exists():
        for vk, fname in view_files.items():
            fpath = target_folder / fname
            if fpath.exists():
                try:
                    images[vk] = PILImage.open(fpath)
                except Exception:
                    images[vk] = None
            else:
                images[vk] = None

    # Load metadata.json if available
```

```
    meta = {}
    if target_folder.exists() and (target_folder / "metadata.json").exists():
        try:
            meta = json.loads((target_folder / "metadata.json").read_text())
        except Exception:
            pass

    # Pre-populate defaults:
    # 1. User given from metadata/Excel: Vein, Mole, Skin texture
    user_vein = row_data.get("Doctor Vein Visibility") or meta.get("doctor_verified_features", {}).get("vein_visibility") o
    user_mole = row_data.get("Doctor Mole Presence") or meta.get("doctor_verified_features", {}).get("mole") or "No"
    user_skin = row_data.get("Doctor Skin Texture") or meta.get("doctor_verified_features", {}).get("skin_texture") or "Smo

    # 2. Defaults for remaining 4
    doc_sc = row_data.get("Doctor Skin Color") or "Medium"
    doc_pc = row_data.get("Doctor Palm Color") or "PinkWhite"
    doc_sh = row_data.get("Doctor Hair Thickness") or "Thin"
    doc_shc = row_data.get("Doctor Hair Color") or "Brown"
    doc_dosha = row_data.get("Doctor Verified Prakriti (Ground Truth)") or row_data.get("ML Predicted Dosha") or "K-P"
    doc_notes = row_data.get("Doctor Clinical Notes") or ""

    return {
        "row_idx": row_idx,
        "id": subj_id,
        "name": row_data.get("Participant Name") or "Unknown",
        "age": row_data.get("Age") or 25,
        "sex": row_data.get("Sex") or ("F" if "female" in subj_id.lower() else "M"),
        "images": images,
        "ml_predicted_dosha": row_data.get("ML Predicted Dosha") or "K-P",
        "ml_confidence_str": row_data.get("ML Confidence / Percentages") or "",
        "defaults": {
            "dosha": doc_dosha,
            "vein": user_vein,
            "mole": user_mole,
            "skin_texture": user_skin,
            "skin_color": doc_sc,
            "palm_color": doc_pc,
            "hair_thickness": doc_sh,
            "hair_color": doc_shc,
            "notes": doc_notes
        }
    }


def save_doctor_verification(subj_id: str, verified_dosha: str, features: dict, clinical_notes: str = ""):
    """
    Performs 3-Way Synchronization:
      1. Master Excel (root & dataset/)
      2. Local Folder (dataset/Wrist_Dataset/{id}/metadata.json)
      3. Supabase Cloud (wrist_uploads table)
    """
    # 1. Update Master Excel Files
    for path in [DATASET_MASTER_XLSX, MASTER_XLSX]:
        if not path.exists():
            continue
        wb = openpyxl.load_workbook(path)
        ws = wb.active
        headers = [ws.cell(1, c).value for c in range(1, ws.max_column + 1)]
        col_map = {name: c for c, name in enumerate(headers, 1) if name}

        fill_verified = PatternFill(start_color="F2F8F2", end_color="F2F8F2", fill_type="solid")
        f_bold = Font(name="Calibri", size=10, bold=True)

        for r in range(2, ws.max_row + 1):
            uid = str(ws.cell(r, col_map["Participant ID"]).value or "").strip()
            if uid.lower() == str(subj_id).strip().lower():
                # Status & Ground Truth
                ws.cell(r, col_map["Validation Status"], value="Verified (Clinical Ground Truth)")
```

```
            ws.cell(r, col_map["Doctor Verified Prakriti (Ground Truth)"], value=verified_dosha)
            ws.cell(r, col_map["Doctor Clinical Notes"], value=clinical_notes or "Clinically verified by Doctor")

            # 7 Clinical Markers
            ws.cell(r, col_map["Doctor Vein Visibility"], value=features.get("vein_visibility"))
            ws.cell(r, col_map["Doctor Mole Presence"], value=features.get("mole"))
            ws.cell(r, col_map["Doctor Skin Texture"], value=features.get("skin_texture"))
            ws.cell(r, col_map["Doctor Skin Color"], value=features.get("skin_color"))
            ws.cell(r, col_map["Doctor Palm Color"], value=features.get("palm_color"))
            ws.cell(r, col_map["Doctor Hair Thickness"], value=features.get("hair_thickness"))
            ws.cell(r, col_map["Doctor Hair Color"], value=features.get("hair_color"))

            # 19 One-hot columns
            sc = features.get("skin_color")
            ws.cell(r, col_map["SC_Fair"], value=1 if sc == "Fair" else 0)
            ws.cell(r, col_map["SC_Medium"], value=1 if sc == "Medium" else 0)
            ws.cell(r, col_map["SC_Dark"], value=1 if sc == "Dark" else 0)

            st = features.get("skin_texture")
            ws.cell(r, col_map["ST_Rough"], value=1 if st == "Rough" else 0)
            ws.cell(r, col_map["ST_Smooth"], value=1 if st == "Smooth" else 0)
            ws.cell(r, col_map["ST_Dry"], value=1 if st == "Dry" else 0)
            ws.cell(r, col_map["ST_Oily"], value=1 if st == "Oily" else 0)

            sh = features.get("hair_thickness")
            ws.cell(r, col_map["SH_Thick"], value=1 if sh == "Thick" else 0)
            ws.cell(r, col_map["SH_Thin"], value=1 if sh == "Thin" else 0)
            ws.cell(r, col_map["SH_Med"], value=1 if sh == "Med" else 0)

            shc = features.get("hair_color")
            ws.cell(r, col_map["SHC_Brown"], value=1 if shc == "Brown" else 0)
            ws.cell(r, col_map["SHC_Black"], value=1 if shc == "Black" else 0)

            ws.cell(r, col_map["Mole on skin"], value=1 if features.get("mole") == "Yes" else 0)

            vv = features.get("vein_visibility")
            ws.cell(r, col_map["VV_Low"], value=1 if vv == "Low" else 0)
            ws.cell(r, col_map["VV_Med"], value=1 if vv == "Med" else 0)
            ws.cell(r, col_map["VV_High"], value=1 if vv == "High" else 0)

            pc = features.get("palm_color")
            ws.cell(r, col_map["PC_PinkRed"], value=1 if pc == "PinkRed" else 0)
            ws.cell(r, col_map["PC_PinkWhite"], value=1 if pc == "PinkWhite" else 0)
            ws.cell(r, col_map["PC_Brown"], value=1 if pc == "Brown" else 0)

            for c in range(1, ws.max_column + 1):
                ws.cell(r, c).fill = fill_verified
            ws.cell(r, col_map["Validation Status"]).font = f_bold
            ws.cell(r, col_map["Doctor Verified Prakriti (Ground Truth)"]).font = f_bold
            break
    wb.save(path)

# 2. Update Local Folder metadata.json
target_folder = WRIST_DIR / str(subj_id).strip().lower()
if not target_folder.exists():
    for d in WRIST_DIR.iterdir():
        if d.is_dir() and d.name.lower() == str(subj_id).strip().lower():
            target_folder = d
            break

if target_folder.exists():
    meta_file = target_folder / "metadata.json"
    meta = {}
    if meta_file.exists():
        try:
            meta = json.loads(meta_file.read_text())
        except Exception:
            pass
```

```
        meta["validation_status"] = "Verified (Clinical Ground Truth)"
        meta["doctor_verified_prakriti"] = verified_dosha
        meta["dominant_dosha"] = verified_dosha[0].upper()
        meta["subdominant_dosha"] = verified_dosha.split("-")[1].upper() if "-" in verified_dosha else None
        meta["doctor_verified_features"] = features
        meta["doctor_notes"] = clinical_notes or "Clinically verified by Doctor"
        meta["is_duplicate"] = False
        meta_file.write_text(json.dumps(meta, indent=4))

    # 3. Update Supabase Cloud
    if supabase_client:
        gender = "female" if "female" in subj_id.lower() else "male"
        payload = {
            "user_id": subj_id,
            "gender": gender,
            "angle": "doctor_ground_truth_portal",
            "predicted_dosha": verified_dosha,
            "confidence": 1.0,
            "image_data": json.dumps({
                "validation_status": "Verified (Clinical Ground Truth)",
                "doctor_verified_prakriti": verified_dosha,
                "dominant_dosha": verified_dosha[0].upper(),
                "subdominant_dosha": verified_dosha.split("-")[1].upper() if "-" in verified_dosha else None,
                "clinical_features": features,
                "doctor_notes": clinical_notes,
                "source": "Doctor Clinical Portal"
            })
        }
        try:
            supabase_client.table("wrist_uploads").insert(payload).execute()
        except Exception as e:
            print(f"[Doctor Portal Supabase Error]: {e}")

    return True


def retrain_model_pipeline(progress_callback=None):
    """
    Retrains the unified ML pipeline on all Verified records.
    Uses cached feature vectors for instant retraining.
    """
    if progress_callback: progress_callback(0.1, "Reading verified clinical cohort...")

    wb = openpyxl.load_workbook(MASTER_XLSX, data_only=True)
    ws = wb.active
    headers = [ws.cell(1, c).value for c in range(1, ws.max_column + 1)]
    col_map = {name: c for c, name in enumerate(headers, 1) if name}

    # Load cache if available
    feature_cache = {}
    if FEATURE_CACHE_PATH.exists():
        try:
            feature_cache = joblib.load(FEATURE_CACHE_PATH)
        except Exception:
            feature_cache = {}

    X_list = []
    y_list = []
    subj_list = []

    # Gather all verified records
    verified_rows = []
    for r in range(2, ws.max_row + 1):
        status = str(ws.cell(r, col_map["Validation Status"]).value or "").strip().lower()
        if "duplicate" in status:
            continue
        if "verified" in status:
            uid = str(ws.cell(r, col_map["Participant ID"]).value).strip()
```

```
        age = ws.cell(r, col_map["Age"]).value or 25
        sex = str(ws.cell(r, col_map["Sex"]).value or "M").strip()
        dosha = str(ws.cell(r, col_map["Doctor Verified Prakriti (Ground Truth)"]).value or "").strip()
        if dosha and dosha[0].upper() in ["K", "P", "V"]:
            verified_rows.append({
                "id": uid,
                "age": float(age),
                "is_male": 1.0 if sex.upper().startswith("M") else 0.0,
                "label": dosha[0].upper() # Dominant dosha
            })

n_samples = len(verified_rows)
if n_samples < 20:
    return {"success": False, "error": f"Not enough verified records (found {n_samples})"}

if progress_callback: progress_callback(0.3, f"Extracting/loading 70-D features for {n_samples} verified subjects...")

cache_updated = False
for i, item in enumerate(verified_rows):
    sid = item["id"]
    if sid in feature_cache:
        feat_vec = feature_cache[sid]
    else:
        # Extract from images
        target_folder = WRIST_DIR / sid.lower()
        if not target_folder.exists():
            for d in WRIST_DIR.iterdir():
                if d.is_dir() and d.name.lower() == sid.lower():
                    target_folder = d
                    break

        view_files = {
            "palm_open_front": target_folder / "palm_open_front.jpg",
            "palm_open_back": target_folder / "palm_open_back.jpg",
            "palm_closed_front": target_folder / "palm_close_front.jpg",
            "palm_closed_back": target_folder / "palm_close_back.jpg"
        }
        images = {}
        for vk, fp in view_files.items():
            images[vk] = PILImage.open(fp) if fp.exists() else None

        feat_vec = extract_participant_features(images, age=item["age"], sex_m=item["is_male"])
        feature_cache[sid] = feat_vec
        cache_updated = True

    X_list.append(feat_vec)
    y_list.append(item["label"])

if cache_updated:
    joblib.dump(feature_cache, FEATURE_CACHE_PATH)

X = np.array(X_list, dtype=float)
y = np.array(y_list)

if progress_callback: progress_callback(0.6, "Fitting class-weighted RBF-SVM & ExtraTrees pipelines...")

# Define best pipeline: StandardScaler + class_weight='balanced' SVC(probability=True)
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", SVC(C=2.0, kernel="rbf", gamma="scale", class_weight="balanced", probability=True, random_state=42))
])

# 5-Fold Stratified Cross Validation
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
bal_accs = cross_val_score(pipe, X, y, cv=skf, scoring="balanced_accuracy")
raw_accs = cross_val_score(pipe, X, y, cv=skf, scoring="accuracy")
f1s = cross_val_score(pipe, X, y, cv=skf, scoring="f1_macro")
# Fit final pipeline on 100% of verified data
```

```
if progress_callback: progress_callback(0.9, "Fitting final production bundle...")
pipe.fit(X, y)

# Save to disk
bundle = {
    "pipeline": pipe,
    "model_name": "Unified RBF-SVM (Balanced)",
    "classes": ["K", "P", "V"],
    "feature_names": FEATURE_NAMES,
    "view_keys": VIEW_KEYS,
    "n_verified_train": n_samples,
    "metrics": {
        "balanced_accuracy": float(np.mean(bal_accs)),
        "accuracy": float(np.mean(raw_accs)),
        "macro_f1": float(np.mean(f1s))
    }
}
joblib.dump(bundle, MODEL_PATH)

if progress_callback: progress_callback(1.0, "Model successfully updated and deployed!")

return {
    "success": True,
    "n_samples": n_samples,
    "class_distribution": pd.Series(y).value_counts().to_dict(),
    "balanced_accuracy": float(np.mean(bal_accs)),
    "accuracy": float(np.mean(raw_accs)),
    "macro_f1": float(np.mean(f1s))
}
```

mini/supabasehelper.py

// Cloud Database & Storage API Helper (Supabase Postgres & Google Gemini Vision)

```
"""
Supabase helper functions for uploading wrist images and data
"""
import io
from PIL import Image
from supabase import create_client
from datetime import datetime
import hashlib
import streamlit as st


# Supabase configuration
SUPABASE_URL = None
SUPABASE_KEY = None
supabase_client = None

def _load_secrets():
    global SUPABASE_URL, SUPABASE_KEY, supabase_client
    # 1. Try Streamlit secrets
    try:
        SUPABASE_URL = st.secrets["supabase"]["url"]
        SUPABASE_KEY = st.secrets["supabase"]["key"]
    except Exception:
        pass

    # 2. Try parsing secrets.toml file directly
    if not SUPABASE_URL or not SUPABASE_KEY:
        try:
            import tomllib
            from pathlib import Path
            current_dir = Path(__file__).resolve().parent
            toml_paths = [
                current_dir / ".streamlit" / "secrets.toml",
                current_dir.parent / ".streamlit" / "secrets.toml",
                Path.cwd() / ".streamlit" / "secrets.toml",
                Path.cwd() / "mini" / ".streamlit" / "secrets.toml"
            ]
            for path in toml_paths:
                if path.exists():
                    with open(path, "rb") as f:
                        data = tomllib.load(f)
                        if "supabase" in data:
                            SUPABASE_URL = data["supabase"].get("url")
                            SUPABASE_KEY = data["supabase"].get("key")
                            break
        except Exception:
            pass

    if SUPABASE_URL and SUPABASE_KEY:
        try:
            supabase_client = create_client(SUPABASE_URL, SUPABASE_KEY)
        except Exception as e:
            print(f"[Supabase Helper] Failed to initialize client: {e}")

_load_secrets()

def get_gemini_api_key():
    """Get Gemini API Key from Streamlit secrets or secrets.toml file directly"""
    # 1. Try from Streamlit secrets
    try:
        if "supabase" in st.secrets and "gemini_api_key" in st.secrets["supabase"]:
            return st.secrets["supabase"]["gemini_api_key"]
        if "gemini_api_key" in st.secrets:
            return st.secrets["gemini_api_key"]
```

```
    except Exception:
        pass

    # 2. Try parsing secrets.toml file directly
    try:
        import tomllib
        from pathlib import Path
        current_dir = Path(__file__).resolve().parent
        toml_paths = [
            current_dir / ".streamlit" / "secrets.toml",
            current_dir.parent / ".streamlit" / "secrets.toml",
            Path.cwd() / ".streamlit" / "secrets.toml",
            Path.cwd() / "mini" / ".streamlit" / "secrets.toml"
        ]
        for path in toml_paths:
            if path.exists():
                with open(path, "rb") as f:
                    data = tomllib.load(f)
                    if "supabase" in data and "gemini_api_key" in data["supabase"]:
                        return data["supabase"]["gemini_api_key"]
                    if "gemini_api_key" in data:
                        return data["gemini_api_key"]
    except Exception:
        pass
    return None


def get_groq_api_key():
    """Get Groq API Key from Streamlit secrets or secrets.toml file directly"""
    # 1. Try from Streamlit secrets
    try:
        if "supabase" in st.secrets:
            for k, v in st.secrets["supabase"].items():
                if k.lower() == "groq_api_key":
                    return v
        for k, v in st.secrets.items():
            if k.lower() == "groq_api_key":
                return v
    except Exception:
        pass

    # 2. Try parsing secrets.toml file directly
    try:
        import tomllib
        from pathlib import Path
        current_dir = Path(__file__).resolve().parent
        toml_paths = [
            current_dir / ".streamlit" / "secrets.toml",
            current_dir.parent / ".streamlit" / "secrets.toml",
            Path.cwd() / ".streamlit" / "secrets.toml",
            Path.cwd() / "mini" / ".streamlit" / "secrets.toml"
        ]
        for path in toml_paths:
            if path.exists():
                with open(path, "rb") as f:
                    data = tomllib.load(f)
                    # Check inside [supabase] table case-insensitively
                    if "supabase" in data:
                        for k, v in data["supabase"].items():
                            if k.lower() == "groq_api_key":
                                return v
                    # Check at root level case-insensitively
                    for k, v in data.items():
                        if k.lower() == "groq_api_key":
                            return v
    except Exception:
        pass
    return None
```

```
def compress_image(image, max_size_kb=200, quality=85):
    """
    Compress PIL Image to target size

    Args:
        image: PIL Image object
        max_size_kb: Maximum size in KB (default 200KB)
        quality: Initial JPEG quality (default 85)

    Returns:
        bytes: Compressed image bytes
    """
    # Convert to RGB if necessary
    if image.mode in ('RGBA', 'P'):
        image = image.convert('RGB')

    # Start with initial quality
    output = io.BytesIO()
    image.save(output, format='JPEG', quality=quality, optimize=True)

    # Reduce quality if still too large
    while output.tell() > max_size_kb * 1024 and quality > 20:
        output = io.BytesIO()
        quality -= 5
        image.save(output, format='JPEG', quality=quality, optimize=True)

    return output.getvalue()


def upload_image_to_storage(image, user_id, gender, angle_key):
    """
    Upload a single image to Supabase storage

    Args:
        image: PIL Image object
        user_id: User ID (e.g., "1001")
        gender: Gender ("male" or "female")
        angle_key: Angle key (e.g., "palm_open_front")

    Returns:
        dict: {success: bool, url: str or None, error: str or None}
    """
    try:
        # Compress image
        img_bytes = compress_image(image, max_size_kb=200, quality=85)

        # Create storage path: {id}_{gender}/{angle}.jpg
        folder_name = f"{user_id}_{gender}"
        filename = f"{angle_key}.jpg"
        storage_path = f"{folder_name}/{filename}"

        # Upload to storage bucket
        response = supabase_client.storage.from_("wrist_images").upload(
            storage_path,
            img_bytes,
            {"content-type": "image/jpeg", "upsert": "true"}
        )

        # Check for errors
        if hasattr(response, 'error') and response.error:
            return {
                "success": False,
                "url": None,
                "error": f"Storage error: {response.error.message}"
            }

        # Get public URL
        public_url = supabase_client.storage.from_("wrist_images").get_public_url(storage_path)
```

```
        return {
            "success": True,
            "url": public_url,
            "error": None,
            "size_kb": len(img_bytes) / 1024
        }

    except Exception as e:
        return {
            "success": False,
            "url": None,
            "error": str(e)
        }


def upload_all_images(images_dict, user_id, gender):
    """
    Upload all 4 wrist images to Supabase storage

    Args:
        images_dict: Dict with keys like "palm_open_front" and PIL Image values
        user_id: User ID
        gender: Gender

    Returns:
        dict: {success: bool, urls: dict, errors: list}
    """
    results = {
        "success": True,
        "urls": {},
        "errors": []
    }

    for angle_key, image in images_dict.items():
        result = upload_image_to_storage(image, user_id, gender, angle_key)

        if result["success"]:
            results["urls"][angle_key] = result["url"]
        else:
            results["success"] = False
            results["errors"].append(f"{angle_key}: {result['error']}")

    return results


def insert_wrist_data(user_id, gender, probabilities, final_dosha, confidence, metadata_dict=None):
    """
    Insert wrist analysis data into Supabase table

    Args:
        user_id: User ID
        gender: Gender
        probabilities: Dict with "Vata", "Pitta", "Kapha" probabilities
        final_dosha: Final predicted dosha string (e.g., "Vata", "Vata-Pitta")
        confidence: Confidence score (0-1)
        metadata_dict: Dict with full clinical features, dominant/subdominant doshas, and doctor mode

    Returns:
        dict: {success: bool, record_id: int or None, error: str or None}
    """
    try:
        import json
        image_data_str = json.dumps(metadata_dict) if metadata_dict else None

        # Prepare data for insertion
        data = {
            "user_id": str(user_id),
            "gender": gender,
```

```
            "angle": "combined_4_views",
            "vata_prob": round(probabilities.get("Vata", 0.0), 4),
            "pitta_prob": round(probabilities.get("Pitta", 0.0), 4),
            "kapha_prob": round(probabilities.get("Kapha", 0.0), 4),
            "predicted_dosha": final_dosha,
            "confidence": round(confidence, 4),
            "image_data": image_data_str,
            "image_size": None
        }

        # Insert into table
        response = supabase_client.table("wrist_uploads").insert(data).execute()

        # Check for successful insert
        if response.data and len(response.data) > 0:
            record_id = response.data[0].get("id")
            return {
                "success": True,
                "record_id": record_id,
                "error": None
            }
        else:
            return {
                "success": False,
                "record_id": None,
                "error": "No data returned from insert"
            }

    except Exception as e:
        return {
            "success": False,
            "record_id": None,
            "error": str(e)
        }


def update_wrist_data_prakriti(user_id, predicted_dosha, probabilities, confidence):
    """
    Update predicted dosha and probabilities for an existing user_id record in Supabase
    """
    try:
        data = {
            "predicted_dosha": predicted_dosha,
            "vata_prob": round(probabilities.get("Vata", 0.0), 4),
            "pitta_prob": round(probabilities.get("Pitta", 0.0), 4),
            "kapha_prob": round(probabilities.get("Kapha", 0.0), 4),
            "confidence": round(confidence, 4)
        }

        response = supabase_client.table("wrist_uploads").update(data).eq("user_id", str(user_id)).execute()

        if response.data and len(response.data) > 0:
            return {
                "success": True,
                "error": None
            }
        else:
            return {
                "success": False,
                "error": f"No records updated for user_id '{user_id}'"
            }
    except Exception as e:
        return {
            "success": False,
            "error": str(e)
        }
def save_to_local_dataset(user_id, gender, images_dict, metadata_dict):
    """
```

```
    Saves high-res photos and metadata.json to dataset/Wrist_Dataset/{user_id}_{gender}/
    """
    try:
        import json
        from pathlib import Path
        project_root = Path(__file__).resolve().parent.parent
        target_dir = project_root / "dataset" / "Wrist_Dataset" / f"{user_id}_{gender}"
        target_dir.mkdir(parents=True, exist_ok=True)

        view_filenames = {
            "palm_open_front": "palm_open_front.jpg",
            "palm_open_back": "palm_open_back.jpg",
            "palm_closed_front": "palm_close_front.jpg",
            "palm_closed_back": "palm_close_back.jpg"
        }

        for angle_key, img in images_dict.items():
            fname = view_filenames.get(angle_key, f"{angle_key}.jpg")
            img.save(target_dir / fname, "JPEG", quality=95)

        meta_file = target_dir / "metadata.json"
        meta_file.write_text(json.dumps(metadata_dict, indent=4))
        return True
    except Exception as e:
        print(f"[Local Save Error]: {e}")
        return False


def append_to_master_excel(user_id, gender, age, images_dict, final_dosha, probabilities, clinical_markers, is_doctor_mode=
    """
    Appends a new formatted row with embedded thumbnails to doctor_validation_prakriti_master.xlsx
    """
    try:
        import openpyxl
        from openpyxl.styles import PatternFill, Font, Alignment, Border, Side
        from openpyxl.drawing.image import Image as OpenpyxlImage
        from pathlib import Path
        import io

        project_root = Path(__file__).resolve().parent.parent
        xlsx_paths = [
            project_root / "dataset" / "doctor_validation_prakriti_master.xlsx",
            project_root / "doctor_validation_prakriti_master.xlsx"
        ]

        for xlsx_path in xlsx_paths:
            if not xlsx_path.exists():
                continue
            wb = openpyxl.load_workbook(xlsx_path)
            ws = wb.active

            headers = [ws.cell(1, c).value for c in range(1, ws.max_column + 1)]
            col_map = {name: c for c, name in enumerate(headers, 1) if name}

            new_row = ws.max_row + 1
            ws.row_dimensions[new_row].height = 80

            # Demographics
            ws.cell(new_row, col_map["Participant ID"], value=f"{user_id}_{gender}")
            ws.cell(new_row, col_map["Participant Name"], value="Web App Submission")
            ws.cell(new_row, col_map["Age"], value=age)
            ws.cell(new_row, col_map["Sex"], value="F" if gender.lower() == "female" else "M")

            # Add thumbnails
            view_cols = {
                "palm_open_front": col_map.get("Palm Open Front (Palm)"),
                "palm_open_back": col_map.get("Palm Open Back (Skin/Veins)"),
                "palm_closed_front": col_map.get("Palm Closed Front (Fist)"),
```

```
                "palm_closed_back": col_map.get("Palm Closed Back (Outer Fist)")
            }

            for angle, col_idx in view_cols.items():
                if col_idx and angle in images_dict:
                    try:
                        img = images_dict[angle].copy()
                        img.thumbnail((90, 80))
                        buf = io.BytesIO()
                        img.convert("RGB").save(buf, format="JPEG", quality=85)
                        buf.seek(0)
                        ox_img = OpenpyxlImage(buf)
                        ox_img.width = img.width
                        ox_img.height = img.height
                        col_letter = openpyxl.utils.get_column_letter(col_idx)
                        ws.add_image(ox_img, f"{col_letter}{new_row}")
                    except Exception:
                        pass

            # Status and Predictions
            status = "Verified (Doctor-in-the-Loop Mode)" if is_doctor_mode else "Pending Doctor Validation"
            ws.cell(new_row, col_map["Validation Status"], value=status)
            ws.cell(new_row, col_map["ML Predicted Dosha"], value=final_dosha)

            k_pct = int(probabilities.get("Kapha", 0) * 100)
            p_pct = int(probabilities.get("Pitta", 0) * 100)
            v_pct = int(probabilities.get("Vata", 0) * 100)
            ws.cell(new_row, col_map["ML Confidence / Percentages"], value=f"k-{k_pct}%, p-{p_pct}%, v-{v_pct}%")

            # Clinical features
            if clinical_markers:
                ws.cell(new_row, col_map["Doctor Vein Visibility"], value=clinical_markers.get("vein_visibility"))
                ws.cell(new_row, col_map["Doctor Mole Presence"], value=clinical_markers.get("mole"))
                ws.cell(new_row, col_map["Doctor Skin Texture"], value=clinical_markers.get("skin_texture"))
                ws.cell(new_row, col_map["Doctor Skin Color"], value=clinical_markers.get("skin_color"))
                ws.cell(new_row, col_map["Doctor Palm Color"], value=clinical_markers.get("palm_color"))
                ws.cell(new_row, col_map["Doctor Hair Thickness"], value=clinical_markers.get("hair_thickness"))
                ws.cell(new_row, col_map["Doctor Hair Color"], value=clinical_markers.get("hair_color"))

            wb.save(xlsx_path)
        return True
    except Exception as e:
        print(f"[Excel Append Error]: {e}")
        return False


def upload_complete_analysis(user_id, gender, images_dict, probabilities, final_dosha, confidence,
                             age=None, top_class=None, secondary_class=None, effective_markers=None, is_doctor_mode=False,
    """
    Complete unified upload & save workflow across all 3 platforms:
      1. Supabase Storage (images) + Supabase Database (record + full metadata JSON)
      2. Local Folder (dataset/Wrist_Dataset/{id}/: 4 .jpg images + metadata.json)
      3. Master Excel (doctor_validation_prakriti_master.xlsx: appends row with thumbnails)
    """
    results = {
        "success": False,
        "record_id": None,
        "image_urls": {},
        "errors": []
    }

    # Extract clean clinical markers
    clean_markers = {}
    if effective_markers:
        for k, v in effective_markers.items():
            clean_markers[k.lower().replace(" ", "_")] = v[0] if isinstance(v, (tuple, list)) else v

    # Build complete metadata dict
```

```
dual_dosha = f"{top_class}-{secondary_class}" if secondary_class and secondary_class != top_class else top_class
metadata_dict = {
    "user_id": str(user_id),
    "gender": gender,
    "age": age,
    "ml_predicted_dosha": final_dosha,
    "dominant_dosha": top_class,
    "subdominant_dosha": secondary_class,
    "dual_dosha_label": dual_dosha,
    "probabilities": probabilities,
    "confidence": confidence,
    "clinical_features": clean_markers,
    "is_doctor_mode": is_doctor_mode,
    "validation_status": "Verified (Doctor-in-the-Loop Mode)" if is_doctor_mode else "Pending Doctor Validation",
    "source": "Streamlit Frontend Web App"
}

# 1. Upload images to Supabase storage if provided
if images_dict:
    upload_result = upload_all_images(images_dict, user_id, gender)
    results["image_urls"] = upload_result.get("urls", {})
    if not upload_result["success"]:
        results["errors"].extend(upload_result.get("errors", []))

# 2. Insert data into Supabase table
insert_result = insert_wrist_data(user_id, gender, probabilities, dual_dosha or final_dosha, confidence, metadata_dict=
if not insert_result["success"]:
    results["errors"].append(f"Database insert failed: {insert_result['error']}")
else:
    results["record_id"] = insert_result["record_id"]

# 3. Save to Local Disk (dataset/Wrist_Dataset/)
if images_dict:
    save_to_local_dataset(user_id, gender, images_dict, metadata_dict)

# 4. Append to Master Excel
append_to_master_excel(user_id, gender, age, images_dict, dual_dosha or final_dosha, probabilities, clean_markers, is_d

results["success"] = True
return results
```

whatsapp_pipeline/pipeline.py

// Multi-Channel WhatsApp Agentic Batch Pipeline (End-to-End Processing Orchestration)

```
import os
import time
from pathlib import Path
from .parser import parse_chat_log, group_by_subject
from .extractor import extract_details
from .view_classifier import classify_hand_views
from .predictor import PrakritiPredictor
from .exporter import generate_excel_report

class WhatsAppAIClassifierPipeline:
    def __init__(self, confidence_threshold=0.0):
        self.predictor = PrakritiPredictor(confidence_threshold=confidence_threshold)

    def process_chat_export(self, chat_txt_path, media_dir_path, output_excel_path, progress_callback=None):
        """
        Runs the end-to-end agentic pipeline to parse the chat, extract details,
        classify images, predict Prakriti, and export the results to Excel.
        """
        chat_txt_path = Path(chat_txt_path)
        media_dir_path = Path(media_dir_path)

        if not chat_txt_path.exists():
            raise FileNotFoundError(f"WhatsApp chat log file not found at: {chat_txt_path}")
        if not media_dir_path.exists():
            raise FileNotFoundError(f"WhatsApp media directory not found at: {media_dir_path}")

        if progress_callback:
            progress_callback(2, "Parsing WhatsApp chat messages...")

        print(f"Starting WhatsApp Chat Parsing: {chat_txt_path.name}")
        messages = parse_chat_log(chat_txt_path)
        print(f"  Parsed {len(messages)} messages chronologically.")

        subjects = group_by_subject(messages)
        print(f"  Identified {len(subjects)} image-grouped subjects from chronological batches.")

        if progress_callback:
            progress_callback(5, f"Identified {len(subjects)} subjects. Beginning analysis...")

        results = []
        total_subjects = max(1, len(subjects))
        for idx, subj in enumerate(subjects):
            if idx > 0:
                time.sleep(0.5)
            sender = subj["sender"]
            images = subj["images"]
            text = subj["associated_text"]

            pct = int(5 + (idx / total_subjects) * 88)
            if progress_callback:
                progress_callback(pct, f"[{idx+1}/{total_subjects}] Processing Subject {idx+1}...")

            # 1. Extract Details from unstructured text
            details = extract_details(text)
            subj_label = details.get("name") if details.get("name") and details["name"] != "Unknown" else f"Subject {idx+1}
            if progress_callback:
                progress_callback(pct, f"[{idx+1}/{total_subjects}] Analyzing {subj_label}...")

            print(f"  Extracted Details -> Name: {details['name']}, Age: {details['age']}, Gender: {details['gender']}, Ski

            # 2. Resolve image paths inside the media directory
            full_image_paths = []
            for img in images:
                # Try exact filename match first
```

```
    img_path = media_dir_path / img
    if img_path.exists():
        full_image_paths.append(str(img_path))
    else:
        # Fallback to case-insensitive match
        found = False
        for child in media_dir_path.iterdir():
            if child.name.lower() == img.lower():
                full_image_paths.append(str(child))
                found = True
                break
        if not found:
            print(f"  [Warning] Attachment file '{img}' not found in media directory.")

# 3. Classify Image Views
view_mapping = {}
if len(full_image_paths) > 0:
    view_mapping = classify_hand_views(full_image_paths)
    print(f"  Classified Views -> { {Path(k).name: v for k, v in view_mapping.items()} }")
else:
    print("  [Warning] No matching image files found for this subject.")

# Restructure image views for predictor mapping
mapped_views = {v: k for k, v in view_mapping.items() if v is not None}

# 4. Extract CV clinical markers from available wrist images
loaded_images = {}
for vk, img_p in mapped_views.items():
    if img_p and Path(img_p).exists():
        try:
            from PIL import Image as PILImage
            loaded_images[vk] = PILImage.open(img_p).convert("RGB")
        except Exception:
            loaded_images[vk] = None

from unified_cv import extract_cv_clinical_markers
cv_markers = extract_cv_clinical_markers(loaded_images)

# Rule: If user has given info in chat text, use it; otherwise fill using CV
combined_markers = dict(details)
for k in ["vein_visibility", "mole", "skin_texture", "skin_type", "skin_color", "palm_color", "hair_thickness",
    val = details.get(k)
    if val and str(val).strip() not in ("Unknown", "None", ""):
        combined_markers[k] = val
    else:
        combined_markers[k] = cv_markers.get(k, "Unknown")

# 5. Predict Prakriti with unified CV + ML
open_front_path = mapped_views.get("palm_open_front")
open_back_path = mapped_views.get("palm_open_back")
closed_front_path = mapped_views.get("palm_closed_front")
closed_back_path = mapped_views.get("palm_closed_back")

prediction = self.predictor.predict_subject(
    age=details["age"],
    gender=details["gender"],
    palm_open_front_path=open_front_path,
    palm_open_back_path=open_back_path,
    palm_closed_front_path=closed_front_path,
    palm_closed_back_path=closed_back_path,
    details=combined_markers
)

if prediction["needs_review"]:
    print(f"  [REVIEW] Flagged for review: {', '.join(prediction['review_reasons'])}")
else:
    print(f"  [RESULT] Extracted Prakriti: {prediction['predicted_dosha']}")
# Compile final subject summary
```

```
    subject_result = {
        "sender": sender,
        "name": details["name"],
        "age": details["age"],
        "gender": details["gender"],
        "skin_type": combined_markers.get("skin_type", "Unknown"),
        "skin_color": combined_markers.get("skin_color", "Unknown"),
        "skin_texture": combined_markers.get("skin_texture", "Unknown"),
        "vein_visibility": combined_markers.get("vein_visibility", "Unknown"),
        "palm_color": combined_markers.get("palm_color", "Unknown"),
        "mole": combined_markers.get("mole", "Unknown"),
        "hair_thickness": combined_markers.get("hair_thickness", "None"),
        "hair_color": combined_markers.get("hair_color", "None"),
        "image_views": mapped_views,
        **prediction
    }
    results.append(subject_result)

# 5. Export results to formatted Excel
if progress_callback:
    progress_callback(96, "Generating formatted Excel batch report...")

print(f"\nGenerating Excel report: {output_excel_path}")
report_path = generate_excel_report(results, output_excel_path)
print(f"Excel report created successfully at: {report_path.resolve()}")

if progress_callback:
    progress_callback(100, f"Completed! Successfully analyzed all {len(results)} subjects.")

return results
```

whatsapp_pipeline/predictor.py

// Multi-Modal Prakriti Predictor & Diagnostic API

```
import joblib
import numpy as np
from pathlib import Path
from PIL import Image

PROJECT_ROOT = Path(__file__).resolve().parent.parent
MINI_PATH = PROJECT_ROOT / "mini"

import sys
if str(MINI_PATH) not in sys.path:
    sys.path.insert(0, str(MINI_PATH))

from unified_cv import (
    extract_participant_features, validate_image_quality, apply_clinical_modulation
)


class PrakritiPredictor:
    """
    Unified Computer Vision + Classical ML Predictor for Ayurvedic Prakriti classification.
    Uses 4-view anatomical ROI extraction and the winning trained ML pipeline.
    """
    def __init__(self, model_path=None, confidence_threshold=0.0):
        self.confidence_threshold = confidence_threshold
        if model_path is None:
            model_path = MINI_PATH / "unified_prakriti_pipeline.joblib"
            if not model_path.exists():
                model_path = MINI_PATH / "model_color.joblib"

        self.model_path = Path(model_path)
        if self.model_path.exists():
            self.bundle = joblib.load(self.model_path)
        else:
            self.bundle = None

    def predict_subject(
        self,
        age,
        gender,
        palm_open_front_path=None,
        palm_open_back_path=None,
        palm_closed_front_path=None,
        palm_closed_back_path=None,
        details=None
    ):
        """
        Runs image quality validation, extracts 4-view CV features, and executes
        the unified ML pipeline.

        Returns:
            dict: {
                'vata_prob': float,
                'pitta_prob': float,
                'kapha_prob': float,
                'predicted_dosha': str,
                'confidence': float,
                'cv_features': dict,
                'needs_review': bool,
                'review_reasons': [str]
            }
        """
        result = {
            "vata_prob": 0.0,
            "pitta_prob": 0.0,
```

```
    "kapha_prob": 0.0,
    "predicted_dosha": "Unknown",
    "confidence": 0.0,
    "cv_features": {},
    "needs_review": False,
    "review_reasons": []
}

# 1. Check for missing demographic details
if age in (None, "Unknown", "") or gender in (None, "Unknown", ""):
    result["needs_review"] = True
    result["review_reasons"].append("Missing Age or Gender details")

# Load available images and validate quality
paths = {
    "palm_open_front": palm_open_front_path,
    "palm_open_back": palm_open_back_path,
    "palm_closed_front": palm_closed_front_path,
    "palm_closed_back": palm_closed_back_path
}

images = {}
for vk, p in paths.items():
    if p and Path(p).exists():
        try:
            img = Image.open(p).convert("RGB")
            images[vk] = img
            # Quality check
            q_res = validate_image_quality(img)
            if not q_res["valid"]:
                result["review_reasons"].extend([f"{vk}: {iss}" for iss in q_res["issues"]])
        except Exception as e:
            images[vk] = None
            result["review_reasons"].append(f"Corrupt image {vk}: {e}")
    else:
        images[vk] = None

# Check critical views
if images.get("palm_open_front") is None:
    result["needs_review"] = True
    result["review_reasons"].append("Missing required palm_open_front image")

if images.get("palm_open_back") is None:
    result["needs_review"] = True
    result["review_reasons"].append("Missing required palm_open_back image")

# 2. Run prediction if model is loaded
if self.bundle is not None:
    try:
        is_male = 1.0 if str(gender).strip().upper().startswith("M") else 0.0
        age_val = float(age) if age not in (None, "Unknown", "") else 35.0

        if "pipeline" in self.bundle:
            # 70-D unified pipeline
            feats = extract_participant_features(images, age=age_val, sex_m=is_male).reshape(1, -1)
            probs_arr = self.bundle["pipeline"].predict_proba(feats)[0]
            classes = self.bundle["classes"]
            prob_dict = {cls: float(p) for cls, p in zip(classes, probs_arr)}
        else:
            # Legacy fallback
            from palm_color import extract_features
            sc_img = images.get("palm_open_back")
            pc_img = images.get("palm_open_front")
            if sc_img and pc_img:
                feats = extract_features(sc_img, pc_img, age_val, is_male).reshape(1, -1)
                probs_arr = self.bundle["model"].predict_proba(feats)[0]
                classes = self.bundle["classes"]
                prob_dict = {cls: float(p) for cls, p in zip(classes, probs_arr)}
```

```
            else:
                prob_dict = {"K": 0.33, "P": 0.33, "V": 0.33}

        # Apply clinical modulation using combined clinical markers (from text or CV)
        if details:
            prob_dict = apply_clinical_modulation(
                prob_dict,
                vein=details.get("vein_visibility", "Auto"),
                mole=details.get("mole", "Auto"),
                skin=details.get("skin_texture") or details.get("skin_type", "Auto"),
                skin_color=details.get("skin_color", "Auto"),
                palm_color=details.get("palm_color", "Auto"),
                hair_thickness=details.get("hair_thickness", "Auto"),
                hair_color=details.get("hair_color", "Auto")
            )

        result["kapha_prob"] = prob_dict.get("K", 0.0)
        result["pitta_prob"] = prob_dict.get("P", 0.0)
        result["vata_prob"] = prob_dict.get("V", 0.0)

        ranked = sorted(prob_dict.items(), key=lambda x: x[1], reverse=True)
        top_class = ranked[0][0]
        sec_class = ranked[1][0] if len(ranked) > 1 else None
        confidence = prob_dict[top_class]

        # Shared / Dual Prakriti (e.g. K-P, K-V, P-V) if secondary dosha is significant (>= 25%)
        if sec_class and ranked[1][1] >= 0.25:
            result["predicted_dosha"] = f"{top_class}-{sec_class}"
        else:
            result["predicted_dosha"] = top_class

        result["confidence"] = confidence
        result["dominant_dosha"] = top_class
        result["secondary_dosha"] = sec_class if (sec_class and ranked[1][1] >= 0.25) else None

        if confidence < self.confidence_threshold:
            result["needs_review"] = True
            result["review_reasons"].append(f"Low confidence ({confidence:.1%} < {self.confidence_threshold:.0%})")

    except Exception as e:
        result["needs_review"] = True
        result["review_reasons"].append(f"Prediction error: {e}")

# Override if ground truth is already provided in text
if details and details.get("prakriti") and details.get("prakriti") != "Unknown":
    result["predicted_dosha"] = details["prakriti"]
    result["confidence"] = 1.0

return result
```

mini/app.py

// Streamlit Clinical Inference Web Interface & Prediction Controller

```
# Trigger hot reload: pipeline fix applied
import streamlit as st
import numpy as np
import joblib
from PIL import Image
import os
from datetime import datetime
import random
from pathlib import Path
from supabasehelper import upload_complete_analysis
from palm_color import extract_features


# ==========================
# Helper function
# ==========================
def generate_user_id():
    """Generate unique user ID using timestamp and random number"""
    timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
    random_num = random.randint(1000, 9999)
    return f"{timestamp}_{random_num}"


# ==========================
# Model definitions
# ==========================
VIEW_ORDER = ["palm_open_front", "palm_open_back", "palm_closed_front", "palm_closed_back"]


# ==========================
# Load model
# ==========================
@st.cache_resource
def load_model(model_name="unified_prakriti_pipeline.joblib"):
    base_dir = Path(__file__).resolve().parent
    model_path = base_dir / model_name
    if not model_path.exists():
        model_path = base_dir / "model_color.joblib"
    return joblib.load(model_path)


# ==========================
# Dosha display metadata
# ==========================
DOSHA_DISPLAY = {
    "K": ("Kapha", "■"),
    "P": ("Pitta", "■"),
    "V": ("Vata", "■■"),
    "K-P": ("Kapha-Pitta", "■■"),
    "K-V": ("Kapha-Vata", "■■■"),
    "P-K": ("Pitta-Kapha", "■■"),
    "P-V": ("Pitta-Vata", "■■■"),
    "V-K": ("Vata-Kapha", "■■■"),
    "V-P": ("Vata-Pitta", "■■■"),
}


def display_name_and_emoji(class_label):
    return DOSHA_DISPLAY.get(class_label, (class_label, "■"))


# ==========================
# Prediction: 4-View Unified CV + ML Pipeline
# ==========================
```

```
def predict_dosha(images_by_angle, age, sex_m, bundle):
    """Extract features across all 4 anatomical views + age/sex and predict dominant dosha."""
    if "pipeline" in bundle:
        from unified_cv import extract_participant_features
        feats = extract_participant_features(images_by_angle, age, sex_m).reshape(1, -1)
        pipeline = bundle["pipeline"]
        probs = pipeline.predict_proba(feats)[0]
        return {cls: float(p) for cls, p in zip(bundle["classes"], probs)}
    else:
        # Fallback for legacy 22-feature bundle
        feats = extract_features(
            images_by_angle[bundle.get("skin_view", "palm_open_back")],
            images_by_angle[bundle.get("palm_view", "palm_open_front")],
            age, sex_m,
        ).reshape(1, -1)
        probs = bundle["model"].predict_proba(feats)[0]
        return {cls: float(p) for cls, p in zip(bundle["classes"], probs)}


# ========================
# Clinical Modulation (Doctor-in-the-Loop)
# ========================
def apply_clinical_modulation(raw_probs, vein="Auto (from images)", mole="Auto (from images)", skin="Auto (from images)"):
    """
    Apply classical Ayurvedic likelihood modulation to CV-extracted probabilities.
    When set to 'Auto (from images)', likelihood multiplier is 1.0 (pure CV).
    When clinical markers are specified by practitioner, modulates V/P/K and renormalizes.
    """
    multipliers = {"V": 1.0, "P": 1.0, "K": 1.0}

    # 1. Vein Visibility
    if "High" in vein:  # Prominent / Superficial (Vata hallmark)
        multipliers["V"] *= 1.45
        multipliers["K"] *= 0.70
    elif "Low" in vein:   # Deep / Concealed (Kapha hallmark)
        multipliers["K"] *= 1.45
        multipliers["V"] *= 0.65
    elif "Medium" in vein:
        multipliers["P"] *= 1.15

    # 2. Mole on Hand / Wrist
    if mole == "Yes":
        multipliers["P"] *= 1.30
        multipliers["V"] *= 1.15
        multipliers["K"] *= 0.85
    elif mole == "No":
        multipliers["K"] *= 1.10
        multipliers["P"] *= 0.95

    # 3. Skin Type / Texture
    if "Dry" in skin:  # Ruksha (Vata hallmark)
        multipliers["V"] *= 1.60
        multipliers["K"] *= 0.60
        multipliers["P"] *= 0.90
    elif "Rough" in skin:  # Khara (Vata hallmark)
        multipliers["V"] *= 1.50
        multipliers["K"] *= 0.65
        multipliers["P"] *= 0.90
    elif "Smooth" in skin or "Normal" in skin:  # Snigdha / Slakshna (Kapha/Pitta)
        multipliers["K"] *= 1.25
        multipliers["P"] *= 1.10
        multipliers["V"] *= 0.80
    elif "Oily" in skin:  # Sneha (Kapha / Pitta)
        multipliers["K"] *= 1.35
        multipliers["P"] *= 1.30
        multipliers["V"] *= 0.60

    # Calculate modulated unnormalized values
```

```
    modulated = {cls: raw_probs.get(cls, 0.0) * multipliers.get(cls, 1.0) for cls in raw_probs}
    total = sum(modulated.values())
    if total > 0:
        return {cls: val / total for cls, val in modulated.items()}
    return raw_probs


# ==========================
# Load demo images
# ==========================
def load_demo_image(image_path):
    """Load demo image from file"""
    try:
        base_dir = Path(__file__).resolve().parent
        resolved_path = base_dir / image_path
        if resolved_path.exists():
            return Image.open(resolved_path)
    except:
        pass
    return None


# ==========================
# Streamlit UI
# ==========================
st.set_page_config(
    page_title="Dosha Classifier",
    page_icon="■■",
    layout="centered",
)

# Sidebar Navigation
st.sidebar.title("Navigation")
page = st.sidebar.radio("Go to", ["Wrist Dosha Analysis", "WhatsApp Batch Import", "■ Doctor Clinical Portal"])

if page == "WhatsApp Batch Import":
    from whatsapp_ui import render_whatsapp_batch_page
    render_whatsapp_batch_page()
    st.stop()

if page == "■ Doctor Clinical Portal":
    from doctor_ui import render_doctor_portal_page
    render_doctor_portal_page()
    st.stop()

st.title("Ayurvedic Wrist Dosha Analysis")
st.markdown(
    "Upload 4 wrist images from different angles. The model reads your skin and "
    "palm colour (with age and sex) to predict your dominant dosha once every view is uploaded."
)

# -------------------------
# Session state
# -------------------------
if "uploaded_images" not in st.session_state:
    st.session_state.uploaded_images = {}

if "upload_counts" not in st.session_state:
    st.session_state.upload_counts = {v: 0 for v in VIEW_ORDER}

if "prediction" not in st.session_state:
    st.session_state.prediction = None

if "raw_cv_prediction" not in st.session_state:
    st.session_state.raw_cv_prediction = None

if "user_info" not in st.session_state:
    st.session_state.user_info = {"user_id": "", "gender": ""}
```

```
if "uploaded_to_supabase" not in st.session_state:
    st.session_state.uploaded_to_supabase = False

if "disclaimer_dismissed" not in st.session_state:
    st.session_state.disclaimer_dismissed = False

# -------------------------
# Disclaimer Dialog
# -------------------------
if not st.session_state.disclaimer_dismissed:
    with st.container(border=True):
        st.markdown("### ■■ Disclaimer")
        st.markdown("""
        This is an experimental AI tool developed for educational and research purposes.

        **Important Points:**
        - This tool provides preliminary indications only and should not be considered medical advice
        - Results may not be clinically accurate
        - Consult a qualified Ayurvedic practitioner for professional dosha assessment
        - Do not make health decisions based solely on these results

        **Image Quality Guidelines:**
        - Take images in good lighting conditions
        - Use a clear, light-colored background
        - Ensure wrist is clearly visible and in focus

        **Data Usage Consent:**
        - Your uploaded wrist images and gender information will be stored securely
        - Data may be used for educational and research purposes to improve the model
        - All data is anonymized and handled in accordance with privacy standards

        **Your Privacy:**
        - Images are compressed and stored securely in cloud storage
        - Only anonymized User IDs are used for data tracking
        """)

        if st.button("Close", type="primary", use_container_width=True):
            st.session_state.disclaimer_dismissed = True
            st.rerun()

# -------------------------
# Load model
# -------------------------
try:
    model_bundle = load_model()
except FileNotFoundError:
    st.error("■ Model file 'model_color.joblib' not found.")
    st.stop()

# -------------------------
# User Information Collection
# -------------------------
st.markdown("### User Information")
gender = st.selectbox(
    "Select Gender",
    options=["", "male", "female"],
    index=0 if not st.session_state.user_info["gender"] else
          (["male", "female"].index(st.session_state.user_info["gender"]) + 1),
    help="Required for analysis"
)

if gender:
    st.session_state.user_info["gender"] = gender
    # Auto-generate user_id if not already set
    if not st.session_state.user_info["user_id"]:
        st.session_state.user_info["user_id"] = generate_user_id()
else:
    st.warning("■■ Please select gender before uploading images")
```

```
    st.stop()

age = st.number_input(
    "Age", min_value=1, max_value=120,
    value=int(st.session_state.user_info.get("age") or 25), step=1,
    help="Used by the model along with the hand images",
)
st.session_state.user_info["age"] = int(age)

# -------------------------
# Optional Clinical Observations (Doctor-in-the-Loop)
# -------------------------
with st.expander("■ Optional Clinical Observations (Doctor-in-the-Loop)", expanded=False):
    st.caption(
        "By default, the classifier automatically relies on the Computer Vision pipeline from your 4 hand images. "
        "If you or an Ayurvedic practitioner specify any of these clinical markers, the model incorporates them "
        "using classical Ayurvedic diagnostic correlations."
    )
    clin_col1, clin_col2, clin_col3 = st.columns(3)
    with clin_col1:
        vein_visibility = st.selectbox(
            "Vein Visibility (Sira)",
            options=["Auto (from images)", "Low (Deep / Concealed)", "Medium", "High (Prominent / Superficial)"],
            index=0,
            help="High prominence indicates Vata; concealed/deep indicates Kapha.",
            key="clin_veins"
        )
        skin_color_in = st.selectbox(
            "Skin Color (Varna)",
            options=["Auto (from images)", "Fair", "Medium", "Dark"],
            index=0,
            help="Skin tone on dorsal hand/wrist.",
            key="clin_sc"
        )
    with clin_col2:
        mole_visibility = st.selectbox(
            "Mole on Hand / Wrist (Tilaka)",
            options=["Auto (from images)", "No", "Yes"],
            index=0,
            help="Presence of moles/freckles typically associates with Pitta/Vata.",
            key="clin_moles"
        )
        palm_color_in = st.selectbox(
            "Palm Color (Kara Tala)",
            options=["Auto (from images)", "Pink-Red", "Pink-White", "Brown"],
            index=0,
            help="Erythema and tone of inner palm.",
            key="clin_pc"
        )
    with clin_col3:
        skin_type = st.selectbox(
            "Skin Texture / Type (Sparsha)",
            options=["Auto (from images)", "Normal / Smooth", "Dry", "Rough", "Oily"],
            index=0,
            help="Dry/Rough skin strongly correlates with Vata; Smooth/Oily with Kapha or Pitta.",
            key="clin_skin"
        )
        hair_thickness_in = st.selectbox(
            "Skin Hair Thickness (Kesha)",
            options=["Auto (from images)", "Thin / Sparse", "Medium", "Thick"],
            index=0,
            help="Forearm/wrist hair density.",
            key="clin_hair"
        )

st.markdown("---")

# -------------------------
```

```
# Wrist View Definitions with Demo Images
# -------------------------
ANGLE_CONFIG = {
    "palm_open_front": {
        "title": "Palm Open – Front View",
        "description": "Hand open, palm facing camera",
        "demo_image": "demo_images/wristfrontopen.jpg",
        "status": "pending"
    },
    "palm_open_back": {
        "title": "Palm Open – Back View",
        "description": "Hand open, back of hand facing camera",
        "demo_image": "demo_images/wristbackopen.jpg",
        "status": "pending"
    },
    "palm_closed_front": {
        "title": "Palm Closed – Front View",
        "description": "Hand closed as fist, knuckles facing camera",
        "demo_image": "demo_images/wristfrontclose.jpg",
        "status": "pending"
    },
    "palm_closed_back": {
        "title": "Palm Closed – Back View",
        "description": "Hand closed as fist, back of hand facing camera",
        "demo_image": "demo_images/wristbackclose.png",
        "status": "pending"
    }
}

# Update status based on what's uploaded
for angle in ANGLE_CONFIG:
    if angle in st.session_state.uploaded_images:
        ANGLE_CONFIG[angle]["status"] = "uploaded"

# -------------------------
# REQUIRED WRIST VIEWS - 2x2 Grid with Demo Images
# -------------------------
st.markdown("### ■ Required Wrist Views")
st.markdown("Upload images for all 4 positions below. Click to expand each section:")

# Create 2x2 grid
grid_col1, grid_col2 = st.columns(2)

# Function to create view section
def create_view_section(angle_key, col):
    config = ANGLE_CONFIG[angle_key]

    # Determine expander state and title
    is_uploaded = config["status"] == "uploaded"
    expander_title = f"■ {config['title']}" if is_uploaded else f"■ {config['title']}"

    with col:
        with st.expander(expander_title, expanded=not is_uploaded):
            if is_uploaded:
                uploaded_img = st.session_state.uploaded_images[angle_key]
                st.success("**Uploaded** ✓")
                st.image(uploaded_img, caption="Your uploaded image", width=150)

                # Image quality validation check
                from unified_cv import validate_image_quality
                q_res = validate_image_quality(uploaded_img)
                if not q_res["valid"]:
                    st.warning("■■ " + "; ".join(q_res["issues"]))
                else:
                    st.caption(f"✓ Quality verified (Sharpness: {q_res['blur_score']:.0f})")

                # Option to re-upload
                if st.button(f"Replace Image", key=f"replace_{angle_key}"):
```

```
                    del st.session_state.uploaded_images[angle_key]
                    st.session_state.raw_cv_prediction = None
                    st.session_state.prediction = None
                    st.session_state.uploaded_to_supabase = False
                    st.rerun()

            else:
                # Show demo image and upload interface
                demo_img = load_demo_image(config["demo_image"])
                if demo_img:
                    st.image(demo_img, caption="Example of this view", width=150)

                st.caption(config["description"])

                # Upload interface
                uploaded_file = st.file_uploader(
                    "Upload your wrist image",
                    type=["jpg", "jpeg", "png"],
                    key=f"upload_{angle_key}_{st.session_state.upload_counts[angle_key]}",
                    label_visibility="collapsed"
                )

                if uploaded_file:
                    image = Image.open(uploaded_file).convert("RGB")
                    st.session_state.uploaded_images[angle_key] = image
                    st.session_state.upload_counts[angle_key] += 1
                    st.session_state.raw_cv_prediction = None
                    st.session_state.prediction = None
                    st.session_state.uploaded_to_supabase = False
                    st.rerun()

# Row 1 - Palm Open Views
create_view_section("palm_open_front", grid_col1)
create_view_section("palm_open_back", grid_col2)

# Row 2 - Palm Closed Views
grid_col3, grid_col4 = st.columns(2)
create_view_section("palm_closed_front", grid_col3)
create_view_section("palm_closed_back", grid_col4)

# Progress indicator
uploaded_count = len(st.session_state.uploaded_images)
if uploaded_count < 4:
    st.progress(uploaded_count / 4, text=f"Progress: {uploaded_count}/4 views uploaded")
    st.caption(f"*{4 - uploaded_count} more view(s) needed before analysis can run*")
else:
    st.success("■ All 4 views uploaded!")

st.markdown("---")

# -------------------------
# Run colour-based prediction once all 4 views are present
# -------------------------
if uploaded_count == 4 and st.session_state.raw_cv_prediction is None:
    with st.spinner("Analyzing anatomical features & extracting 7 clinical markers..."):
        sex_m = 1 if st.session_state.user_info["gender"] == "male" else 0
        from unified_cv import extract_cv_clinical_markers

        # 1. Unified CV feature extraction
        st.session_state.cv_clinical_markers = extract_cv_clinical_markers(st.session_state.uploaded_images)
        st.session_state.raw_cv_prediction = predict_dosha(
            st.session_state.uploaded_images,
            st.session_state.user_info["age"], sex_m, model_bundle,
        )

# Dynamically resolve effective markers and apply clinical modulation
if st.session_state.raw_cv_prediction is not None:
    cv_m = st.session_state.get("cv_clinical_markers", {})
```

```
    # Resolve user overrides vs CV detections
    eff_vein = vein_visibility if vein_visibility != "Auto (from images)" else cv_m.get("vein_visibility", "Low")
    eff_mole = mole_visibility if mole_visibility != "Auto (from images)" else cv_m.get("mole", "No")
    eff_skin = skin_type if skin_type != "Auto (from images)" else cv_m.get("skin_texture", "Smooth")
    eff_sc = skin_color_in if skin_color_in != "Auto (from images)" else cv_m.get("skin_color", "Medium")
    eff_pc = palm_color_in if palm_color_in != "Auto (from images)" else cv_m.get("palm_color", "PinkWhite")
    eff_hair = hair_thickness_in if hair_thickness_in != "Auto (from images)" else cv_m.get("hair_thickness", "Thin")
    eff_hair_c = cv_m.get("hair_color", "Brown")

    is_doctor_mode = any([
        vein_visibility != "Auto (from images)",
        mole_visibility != "Auto (from images)",
        skin_type != "Auto (from images)",
        skin_color_in != "Auto (from images)",
        palm_color_in != "Auto (from images)",
        hair_thickness_in != "Auto (from images)"
    ])

    from unified_cv import apply_clinical_modulation
    st.session_state.prediction = apply_clinical_modulation(
        st.session_state.raw_cv_prediction,
        vein=eff_vein,
        mole=eff_mole,
        skin=eff_skin,
        skin_color=eff_sc,
        palm_color=eff_pc,
        hair_thickness=eff_hair,
        hair_color=eff_hair_c
    )

    st.session_state.effective_markers = {
        "Skin Color": (eff_sc, "Practitioner" if skin_color_in != "Auto (from images)" else "Computer Vision"),
        "Skin Texture": (eff_skin, "Practitioner" if skin_type != "Auto (from images)" else "Computer Vision"),
        "Vein Visibility": (eff_vein, "Practitioner" if vein_visibility != "Auto (from images)" else "Computer Vision"),
        "Mole on Skin": (eff_mole, "Practitioner" if mole_visibility != "Auto (from images)" else "Computer Vision"),
        "Palm Color": (eff_pc, "Practitioner" if palm_color_in != "Auto (from images)" else "Computer Vision"),
        "Hair Thickness": (eff_hair, "Practitioner" if hair_thickness_in != "Auto (from images)" else "Computer Vision"),
        "Hair Color": (eff_hair_c, "Computer Vision")
    }

# -------------------------
# FINAL DOSHA PREDICTION
# -------------------------
if st.session_state.prediction:
    st.markdown("### ■ **Dosha Analysis**")

    probs = st.session_state.prediction
    top_class = max(probs.items(), key=lambda x: x[1])[0]
    confidence = probs[top_class]
    dosha_name, emoji = display_name_and_emoji(top_class)

    # Display final result
    with st.container(border=True):
        cols = st.columns([1, 3])
        with cols[0]:
            st.markdown(f"# {emoji}")
        with cols[1]:
            st.markdown(f"## {dosha_name}")
            mode_badge = "■ Doctor-in-the-Loop Mode" if is_doctor_mode else "■ 100% Automated CV Mode"
            st.caption(f"Confidence: {confidence * 100:.1f}% · {model_bundle.get('model_name', 'Unified ML')} ({mode_badge})")
            if is_doctor_mode:
                active_markers = []
                if vein_visibility != "Auto (from images)":
                    active_markers.append(f"Veins: {vein_visibility}")
                if mole_visibility != "Auto (from images)":
                    active_markers.append(f"Mole: {mole_visibility}")
                if skin_type != "Auto (from images)":
                    active_markers.append(f"Skin: {skin_type}")
```

```
            st.info(f"**Clinical Markers Applied:** {', '.join(active_markers)}")

    # 7 Physical Clinical Diagnostic Features Card
    if "effective_markers" in st.session_state:
        with st.expander("■ Verified Clinical Diagnostic Features (7 Markers)", expanded=True):
            em = st.session_state.effective_markers
            f_cols = st.columns(4)
            for idx, (m_name, (m_val, m_src)) in enumerate(em.items()):
                col_target = f_cols[idx % 4]
                with col_target:
                    src_icon = "■" if m_src == "Practitioner" else "■"
                    st.metric(label=m_name, value=str(m_val), delta=f"{src_icon} {m_src}", delta_color="off")

    # Soft secondary tendency (the 2nd-highest dosha) — NOT a validated dual-dosha claim
    ranked = sorted(probs.items(), key=lambda x: x[1], reverse=True)
    if len(ranked) > 1 and ranked[1][1] >= 0.25:
        sec_name, sec_emoji = display_name_and_emoji(ranked[1][0])
        st.caption(f"Secondary tendency: {sec_name} {sec_emoji} "
                   f"({ranked[1][1]*100:.0f}%) — lower confidence, not a clinical dual-dosha result")

    # All-class probabilities
    st.markdown("##### Full Probability Distribution:")
    for cls, p in sorted(probs.items(), key=lambda x: x[1], reverse=True):
        name, emoji = display_name_and_emoji(cls)
        col1, col2, col3 = st.columns([2, 4, 2])
        with col1:
            st.markdown(f"**{name}** {emoji}")
        with col2:
            st.progress(p)
        with col3:
            st.markdown(f"**{p*100:.1f}%**")

    # Auto-save to cloud if not already uploaded
    if not st.session_state.uploaded_to_supabase:
        if st.session_state.user_info["user_id"] and st.session_state.user_info["gender"]:
            with st.spinner("■ Saving to cloud..."):
                user_id = st.session_state.user_info["user_id"]
                gender = st.session_state.user_info["gender"]
                images_dict = st.session_state.uploaded_images

                # The 3-class model outputs real Vata/Pitta/Kapha probabilities,
                # so the Supabase vata_prob/pitta_prob/kapha_prob columns are
                # populated directly. predicted_dosha stores the dominant dosha.
                dosha_probs = {
                    "Vata": probs.get("V", 0.0),
                    "Pitta": probs.get("P", 0.0),
                    "Kapha": probs.get("K", 0.0),
                }

                result = upload_complete_analysis(
                    user_id=user_id,
                    gender=gender,
                    images_dict=images_dict,
                    probabilities=dosha_probs,
                    final_dosha=f"{dosha_name} ({top_class})",
                    confidence=confidence,
                    age=st.session_state.user_info.get("age", 25),
                    top_class=top_class,
                    secondary_class=ranked[1][0] if len(ranked) > 1 else None,
                    effective_markers=st.session_state.get("effective_markers", {}),
                    is_doctor_mode=is_doctor_mode
                )

                if result["success"]:
                    st.session_state.uploaded_to_supabase = True
                    st.session_state.upload_record_id = result["record_id"]
                    st.rerun()
                else:
```

```
                    st.error(f"■ Cloud save failed: {', '.join(result['errors'])}")

    # Show success messages
    st.balloons()
    st.success("■ Analysis complete - dominant dosha from skin & palm colour!")

    if st.session_state.uploaded_to_supabase:
        st.success(f"■■ Saved to cloud! User ID: **{st.session_state.user_info['user_id']}**")

        # Show reset button only after successful upload
        if st.button("■ Start New Analysis", type="primary", use_container_width=True):
            st.session_state.uploaded_images.clear()
            st.session_state.upload_counts = {v: 0 for v in VIEW_ORDER}
            st.session_state.raw_cv_prediction = None
            st.session_state.prediction = None
            st.session_state.uploaded_to_supabase = False
            st.session_state.user_info = {"user_id": "", "gender": ""}
            st.rerun()

# -------------------------
# INFO SECTION
# -------------------------
with st.expander("■ Understanding Your Dosha Results", expanded=False):
    st.markdown("""
    ### What Are Doshas?

    In Ayurveda, doshas are the three fundamental energies that govern our physical and mental processes.

    **■■ Vata (Air + Ether)**
    - *Qualities:* Dry, light, cold, rough, subtle, mobile
    - *Governs:* Movement, breathing, circulation, nerve impulses
    - *When balanced:* Creative, energetic, adaptable
    - *When imbalanced:* Anxiety, dry skin, constipation, insomnia

    **■ Pitta (Fire + Water)**
    - *Qualities:* Hot, sharp, light, liquid, oily
    - *Governs:* Digestion, metabolism, body temperature
    - *When balanced:* Intelligent, focused, good leadership
    - *When imbalanced:* Anger, inflammation, heartburn, skin rashes

    **■ Kapha (Earth + Water)**
    - *Qualities:* Heavy, slow, cool, oily, smooth, dense
    - *Governs:* Structure, stability, lubrication
    - *When balanced:* Calm, loving, strong, grounded
    - *When imbalanced:* Weight gain, lethargy, congestion, depression

    ### Understanding Your Results

    - **Single-Dosha:** One energy is predominant
    - **Dual-Dosha:** Two energies are jointly predominant (most common)

    *Note: This analysis is based on wrist characteristics and provides an indication of your predominant dosha(s), predict
    """)

st.markdown("---")
st.markdown("*Ayurvedic analysis using AI • Dominant dosha from skin & palm colour + age/sex*")
```